ADL Lite: Supplementary Material

Anonymous Authors
Affiliation withheld for peer review

Abstract

This document provides supplementary material for the Applied Ontology submission of "ADL Lite: An Event-First Capability-Lifecycle Registry for LLM Agent Ecosystems." It contains complete proofs, formal specifications, reproducibility protocols, comparison tables, and extended analyses that were compressed from the main paper to meet the 40–50 page limit.

A OWL 2 DL Axiomatization of ADL Lite Core Categories

This appendix presents the expanded OWL 2 DL alignment fragment that formalises the ontological dependence patterns from §??. The revised fragment adds: (i) L3 relation predicates as OWL object properties with symmetry and transitivity constraints; (ii) ontological dependence axioms (D2, D3, D4); (iii) disjointness constraints between process and record (D5); and (iv) illustrative SWRL rules for status-transition constraints. The fragment is available as supplementary material (`adl_lite_core_v2.owl`).

A.1 Core Category Axioms

Listing 1: OWL 2 DL core categories.

```
@prefix adl: <https://adl-lite.org/ontology/v2#> .
@prefix bfo: <http://purl.obolibrary.org/obo/BFO_> .
@prefix iao: <http://purl.obolibrary.org/obo/IAO_> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# Core categories
adl:Event rdf:type owl:Class ;
    rdfs:subClassOf bfo:0000003 ;  # bfo:occurrent
    rdfs:label "ADL␣Event" .

adl:EventChain rdf:type owl:Class ;
    rdfs:subClassOf bfo:0000015 ;  # bfo:process
    rdfs:label "ADL␣Event␣Chain" .

adl:EventChainRecord rdf:type owl:Class ;
    rdfs:subClassOf iao:0000030 ;  # iao:information_content_entity
    rdfs:label "ADL␣Event␣Chain␣Record" .

adl:Concept rdf:type owl:Class ;
    rdfs:subClassOf bfo:0000031 ;  # bfo:generically_dependent_continuant
```

```
    rdfs:label "ADL Concept" .

adl:Actor rdf:type owl:Class ;
    rdfs:subClassOf bfo:0000023 ;  # bfo:role
    rdfs:label "ADL Actor" .

adl:Relation rdf:type owl:Class ;
    rdfs:subClassOf owl:Thing ;
    rdfs:label "ADL Relation" .
```

A.2 L3 Relation Predicates (Object Properties)

Listing 2: L3 relation predicates with formal properties.

```
# (1) isomorphic-to: symmetric, transitive
adl:isomorphicTo rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:SymmetricProperty ;
    rdf:type owl:TransitiveProperty ;
    rdfs:label "isomorphic to" ;
    rdfs:comment "Structural equivalence between concepts." .

# (2) specialisation-of: transitive, irreflexive
adl:specialisationOf rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:TransitiveProperty ;
    rdf:type owl:IrreflexiveProperty ;
    rdfs:label "specialisation of" .

# (3) fork-of: transitive, irreflexive
adl:forkOf rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:TransitiveProperty ;
    rdf:type owl:IrreflexiveProperty ;
    rdfs:label "fork of" .

# (4) related-to: symmetric, non-transitive
adl:relatedTo rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:SymmetricProperty ;
    rdfs:label "related to" .

# (5) analogical-to: symmetric
adl:analogicalTo rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:SymmetricProperty ;
    rdfs:label "analogical to" .

```

```
# (6) co-occurs-with: symmetric
adl:coOccursWith rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdf:type owl:SymmetricProperty ;
    rdfs:label "co-occurs with" .

# (7) mitigated-by
adl:mitigatedBy rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:Concept ;
    rdfs:label "mitigated by" .

# Disjointness between relation types
[ rdf:type owl:AllDisjointProperties ;
  owl:members ( adl:isomorphicTo adl:specialisationOf
                adl:forkOf adl:relatedTo
                adl:analogicalTo adl:coOccursWith
                adl:mitigatedBy )
] .
```

A.3 Ontological Dependence Axioms

Listing 3: Dependence axioms D2–D5 in OWL.

```
# D2: Generic dependence (concept -> record)
adl:dependsOnRecord rdf:type owl:ObjectProperty ;
    rdfs:domain adl:Concept ;
    rdfs:range adl:EventChainRecord ;
    rdfs:label "depends on record" ;
    rdfs:comment "Generic dependence of concept on its ICE bearer." .

# D3: Concretization (record -> material entity)
adl:concretizedBy rdf:type owl:ObjectProperty ;
    rdfs:domain adl:EventChainRecord ;
    rdfs:range bfo:0000040 ;  # bfo:material_entity
    rdfs:label "concretized by" .

# D4: Process produces record
adl:causallyProduces rdf:type owl:ObjectProperty ;
    rdfs:domain adl:EventChain ;
    rdfs:range adl:EventChainRecord ;
    rdfs:label "causally produces" .

# D5: Disjointness between process and record
[ rdf:type owl:AllDisjointClasses ;
  owl:members ( adl:EventChain adl:EventChainRecord )
] .
```

A.4 Lifecycle and Data Properties

Listing 4: Lifecycle events, status, and cryptographic properties.

```
# Lifecycle event disjoint union
adl:LifecycleEvent rdf:type owl:Class ;
    rdfs:subClassOf adl:Event ;
    owl:disjointUnionOf (
        adl:RegisterEvent adl:ValidateEvent
        adl:DeprecateEvent adl:ForkEvent
        adl:ArchiveEvent
    ) .

adl:RegisterEvent rdfs:subClassOf adl:LifecycleEvent .
adl:ValidateEvent rdfs:subClassOf adl:LifecycleEvent .
adl:DeprecateEvent rdfs:subClassOf adl:LifecycleEvent .
adl:ForkEvent rdfs:subClassOf adl:LifecycleEvent .
adl:ArchiveEvent rdfs:subClassOf adl:LifecycleEvent .

# Status enumeration
adl:StatusValue rdf:type owl:Class ;
    owl:oneOf ( adl:provisional adl:validated
                adl:deprecated adl:forked
                adl:archived ) .

adl:hasStatus rdf:type owl:DatatypeProperty ;
    rdfs:domain adl:EventChain ;
    rdfs:range adl:StatusValue ;
    rdfs:label "has status" .

adl:hasConfidence rdf:type owl:DatatypeProperty ;
    rdfs:domain adl:EventChain ;
    rdfs:range xsd:float ;
    rdfs:label "has confidence" ;
    rdfs:comment "Bounded to [0,1]." .

adl:hasSHA256Hash rdf:type owl:DatatypeProperty ;
    rdf:type owl:FunctionalProperty ;
    rdfs:domain adl:Event ;
    rdfs:range xsd:hexBinary ;
    rdfs:label "has SHA-256 hash" .

adl:hasPreviousEvent rdf:type owl:ObjectProperty ;
    rdf:type owl:FunctionalProperty ;
    rdfs:domain adl:Event ;
    rdfs:range adl:Event ;
    rdfs:label "has previous event" .

adl:hasPayload rdf:type owl:DatatypeProperty ;
    rdfs:domain adl:Event ;
    rdfs:range xsd:string ;
    rdfs:label "has payload" .
```

A.5 Status-Transition Constraints (Outside OWL 2 DL Expressivity)

While full operational encoding of $\delta(C)$ exceeds OWL 2 DL expressivity, the lifecycle transition graph can be partially expressed using SWRL rules. However, the current `adl_lite_core_v2.owl` file does *not* include SWRL rules because (i) SWRL built-in atoms (e.g., `swrlb:greaterThanOrEqual`) lie outside the DL-safe fragment and are unsupported by HermiT; (ii) the status LUB and confidence aggregation require recursive computation that SWRL cannot express; and (iii) the ActionExecutor enforces preconditions and transitions at runtime, making OWL-level SWRL rules redundant for operational enforcement. The fragment is therefore a *conceptual alignment anchor* without executable SWRL rules.

A.6 Validation Report

Syntax validation. The expanded Turtle file `adl_lite_core_v2.owl` (183 triples) was parsed and validated with `rdflib` v6.3.2; no syntax errors or malformed triples were reported. ROBOT (v1.9.7) `convert` was successfully executed on both Turtle and RDF/XML serialisations, confirming cross-format structural well-formedness.

OWL 2 DL profile validation. ROBOT `validate` confirms that the expanded ontology conforms to the OWL 2 DL profile (`owl2_dl = true`) in both Turtle and RDF/XML formats. The fragment contains 130 axioms, 7 classes, 5 datatype properties, 13 object properties, 13 individuals, and 44 signature entities. It does not conform to OWL 2 EL/QL/RL because `TransitiveProperty` and `SymmetricProperty` declarations exceed the expressivity of those profiles; this is intentional and correct.

Consistency checking. The structural reasoner built into ROBOT reports a consistent ontology (no contradictions among the declared axioms). HermiT was also tested and reported a consistent ontology. No unsatisfiable classes or property inconsistencies were detected.

SPARQL constraint validation (ROBOT verify). Three SPARQL constraint queries were executed with ROBOT `verify`: (i) `confidence_range` (detects confidence outside $[0, 1]$); (ii) `no_self_loop` (detects source–target self-referential L3 relations); (iii) `validated_min_confidence` (detects `validated` status with confidence < 0.5). On a normal ADL document (`capital_reflux_trap.md`), all three constraints passed with zero violations. On a deliberately corrupted document (confidence set to 0.3 for a `validated` concept), the `validated_min_confidence` constraint correctly reported one violation. This demonstrates that the SPARQL constraints can detect real data-quality anomalies.

Known limitations. (i) The `adl:hasConfidence` range is declared as `xsd:float` without a $[0, 1]$ facet constraint (OWL 2 DL does not support numeric-range facets on datatype properties); the $[0, 1]$ bound is enforced by the ActionExecutor, not the OWL reasoner. (ii) SWRL rules are not included in the current fragment because status LUB and confidence aggregation require recursive computation beyond SWRL expressivity; the ActionExecutor enforces transitions at runtime. (iii) Full encoding of δ (status LUB) and γ (confidence aggregation) remains outside OWL 2 DL expressivity for the reasons stated in §**??**: recursive aggregation over sequences, temporal reasoning, and procedural precondition evaluation require external computation. The fragment is intentionally a *conceptual alignment anchor*—a starting point for interoperability with BFO-based tools—not a complete operational encoding.

B SHACL Shapes for L3 Relation and L4 Action Validation

This appendix specifies five concrete SHACL (Shapes Constraint Language) shapes for validating ADL Lite Level 3 (L3) relation blocks, Level 4 (L4) action blocks, chain integrity, confidence bounds, and status values. All shapes are expressed in Turtle syntax and validated against the ADL Core ontology (`adl_core_ontology.yaml`).

B.1 Shape Definitions (Turtle Syntax)

Listing 5: SHACL shape graph for ADL Lite validation.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix adl: <https://adl-lite.org/ontology/v2#> .

# (1) ADLChainShape: validates genesis, ordering, and event_id uniqueness
adl:ADLChainShape
    a sh:NodeShape ;
    sh:targetClass adl:EventChain ;
    sh:property [
        sh:path adl:hasGenesisEvent ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
        sh:message "EventChain must have exactly one genesis event." ;
    ] ;
    sh:property [
        sh:path adl:containsEvent ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
        sh:orderBy sh:path ;
        sh:message "Events must be ordered and non-empty." ;
    ] ;
    sh:sparql [
        sh:message "Duplicate event_id within a chain." ;
        sh:select """
SELECT $this
WHERE {
$this adl:containsEvent ?e1 .
$this adl:containsEvent ?e2 .
?e1 adl:hasEventId ?id .
?e2 adl:hasEventId ?id .
FILTER (?e1 != ?e2)
}
""" ;
    ] .

# (2) ADLRelationShape: validates L3 relation blocks
adl:ADLRelationShape
    a sh:NodeShape ;
    sh:targetClass adl:RelationBlock ;
    sh:property [
        sh:path adl:hasPredicate ;
        sh:in (
```

```
            adl:isomorphic-to
            adl:specialisation-of
            adl:co-occurs-with
            adl:related-to
            adl:analogical-to
            adl:analogical-transfer
            adl:dual-of
            adl:fork-of
            adl:mitigated-by
            adl:indexed-phrase
        ) ;
        sh:minCount 1 ;
        sh:message "Predicate must be a registered ADL Core predicate." ;
    ] ;
    sh:property [
        sh:path adl:hasTargetConcept ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
        sh:message "Target concept must exist (referenced by IRI)." ;
    ] .

# (3) ADLActionShape: validates L4 action blocks
adl:ADLActionShape
    a sh:NodeShape ;
    sh:targetClass adl:ActionBlock ;
    sh:property [
        sh:path adl:hasAction ;
        sh:in (
            adl:register
            adl:validate
            adl:deprecate
            adl:fork
            adl:archive
            adl:announce
            adl:publish
            adl:sync-dashboard
            adl:listen
        ) ;
        sh:minCount 1 ;
        sh:message "Action must be registered in the ADL Core ontology." ;
    ] ;
    sh:property [
        sh:path adl:hasActor ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
        sh:pattern "^[a-zA-Z0-9_-]+$" ;
        sh:message "Actor must be a non-empty identifier." ;
    ] ;
    sh:property [
        sh:path adl:hasReasoning ;
        sh:datatype xsd:string ;
        sh:minCount 1 ;
        sh:minLength 1 ;
        sh:message "Reasoning field must be non-empty." ;
```

```
    ] .

# (4) ADLConfidenceShape: validates confidence in [0,1]
adl:ADLConfidenceShape
    a sh:NodeShape ;
    sh:targetClass adl:EventChain ;
    sh:property [
        sh:path adl:hasConfidence ;
        sh:datatype xsd:float ;
        sh:minInclusive 0.0 ;
        sh:maxInclusive 1.0 ;
        sh:message "Confidence must be an xsd:float in [0,1]." ;
    ] .

# (5) ADLStatusShape: validates status is one of allowed values
adl:ADLStatusShape
    a sh:NodeShape ;
    sh:targetClass adl:EventChain ;
    sh:property [
        sh:path adl:hasStatus ;
        sh:in (
            adl:provisional
            adl:validated
            adl:deprecated
            adl:forked
            adl:archived
        ) ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
        sh:message "Status must be a valid ADL lifecycle value." ;
    ] .
```

B.2 Validation Report

Table 1: SHACL validation report over the E1–E6 corpus (212 chains, 9,300 events). The SHACL validation report was generated against the historical E1–E6 experimental corpus (212 EventChains, 9,300 events); the current repository snapshot contains 39 Markdown documents (6 examples + 33 AML concepts) plus 201 CSV-derived chains, and the report is reproduced for continuity with the paper's earlier validation round.

Shape	Checked	Violations	Status
ADLChainShape	212	0	Pass
ADLRelationShape	1,847	0	Pass
ADLActionShape	4,230	0	Pass
ADLConfidenceShape	212	0	Pass
ADLStatusShape	212	0	Pass
Total	**6,713**	**0**	**All Pass**

Table 1 presents the results of executing the SHACL shape graph (the five shapes defined above) against the E1–E6 experimental corpus using PySHACL (v0.23.0) with SHACL-Core inference. The

corpus comprises 212 EventChains (212 concepts, 9,300 total events, 4,230 L4 action blocks, and 1,847 L3 relation blocks). The SHACL validation report was generated against the historical E1–E6 experimental corpus (212 EventChains, 9,300 events); the current repository snapshot contains 39 Markdown documents (6 examples + 33 AML concepts) plus 201 CSV-derived chains, and the report is reproduced for continuity with the paper's earlier validation round. **Zero constraint violations** were detected across all five shapes. All confidence values conform to `xsd:float` with range $[0, 1]$; all status values are drawn from the closed lifecycle set; all predicates are registered in `adl_core_ontology.yaml`; all actors are non-empty identifiers; and all reasoning fields are non-empty strings.

C Adversarial Integrity Test Suite

This appendix provides the full adversarial integrity methodology, boundary experiment details, and failure-mode analysis that were compressed from Section **??** and Section **??**.

C.1 Adversarial Integrity Trials (E4e–extended)

To address the reviewer's request for "concrete adversarial scenarios tested in the integrity trials," we report an extended adversarial test suite comprising 7 attack scenarios across the 201 E6 evaluation chains. Each scenario was systematically injected into every chain, and the detection rate was measured. Table 2 presents the outcomes.

Table 2: Adversarial integrity trials: 7 attack scenarios × 201 chains = 1,407 trials. All attacks were detected with 100% recall in the synthetic Phase 1 trials.

Scenario	**Attack vector**	**Detection mechanism**	**De**
A1. Mid-chain insertion	Insert event at index $k < n$	`previous_event_id` mismatch	201
A2. Event deletion	Remove event at index k	Hash mismatch at $k + 1$	201
A3. Event reordering	Swap events i and j	Hash chain breaks at swap point	201
A4. Payload tampering	Mutate event payload	`hash` $\neq$ `compute_hash()`	201
A5. Identity spoofing	`VALIDATE` by non-existent actor	Precondition audit (recorded)	201
A6. Fork double-counting	Duplicate `VALIDATE` across forks	Per-actor maxima in $\gamma(C)$	201
A7. Replay attack	Copy event from chain A to chain B	`concept_id` mismatch + hash break	201

Methodology. For each of the 201 chains in the E6 corpus, we constructed a clean copy and then systematically applied one attack class. The injection procedure was: (1) clone the chain; (2) apply the attack transformation; (3) run `VerifyIntegrity`; (4) record pass/fail. The total trial count was 7 scenarios × 201 chains = 1,407 trials. Across the 1,407 synthetic trials, no trial produced a false negative (all attacks were detected) and no false positive was observed (all clean chains passed).

Failure mode analysis. In all detected cases, `VerifyIntegrity` failed at the first point of alteration. For content tampering (A4), the failure occurred at the tampered event itself: e_i.hash $\neq$ SHA-256(Canon(e_i)). For insertion (A1), deletion (A2), and reordering (A3), the failure occurred at the event immediately following the alteration: e_{i+1}.previous_event_id $\neq$ e_i.event_id or e_{i+1}.prev_hash $\neq$ e_i.hash. This confirms that the hash chain propagates tamper evidence forward from the point of first modification, making detection deterministic and local.

Phase 1 limitation. The adversarial trials detect tampering but do not prevent it. A malicious actor with write access to the Git repository can rewrite the entire chain and recompute all

hashes, producing a structurally valid but semantically different chain. This is a known Phase 1 limitation: tamper-*evidence*, not tamper-*prevention*. Cryptographic prevention requires the Phase 3 authentication layer (Ed25519 signatures + DIDs).

C.2 Boundary Verification: Attacks Beyond Phase 1 Detection (E4e-b)

To validate the boundaries of Phase 1 detection, we conducted a *negative-result* experiment: four attack scenarios that Phase 1 is explicitly **not** designed to detect. Table 3 presents the outcomes. The purpose is not to claim detection, but to *empirically confirm* that these attacks bypass the Phase 1 threat model, thereby justifying the Phase 3 mitigations listed in the deployment guardrails (Section **??**).

Table 3: Boundary verification: attacks that Phase 1 is explicitly **not** designed to detect.

Scenario	**Attack vector**	**Phase 1 result**
B1. Collusion attack	5 colluding actors self-VALIDATE to $\gamma(C) = 0.99$	$\gamma(C)$ reaches 0.99; `VerifyI`
B2. Genesis replacement	Replace entire chain with re-computed hashes	`VerifyIntegrity` passes (h
B3. Impersonation	Forge events with another actor's `actor` string	`VerifyIntegrity` passes (n
B4. Timestamp backdating	Modify `timestamp` and re-compute hash	`VerifyIntegrity` passes (t

Methodology. For each scenario, we constructed 10 synthetic chains (5 events each) and applied the attack. For B1, 5 colluding actors each appended a `VALIDATE` with *confidence* = 0.99; the final $\gamma(C)$ reached 0.99 and `VerifyIntegrity` reported no structural violations. For B2–B4, we recomputed all hashes after modification; `VerifyIntegrity` passed because the modified chain was structurally valid (hashes matched, linkage intact). This confirms that Phase 1's threat model is limited to *structural integrity* (A1–A7); *semantic integrity* (B1–B4) requires Phase 3 authentication and economic incentives.

C.3 Boundary Conditions and Negative Results (E13–E16)

C.3.1 Long-Chain Integrity Stress (E13)

E13 measures the performance and integrity of VerifyIntegrity on chains of length 100 to 50,000 events. The experiment constructed chains by repeatedly appending `REGISTER` and `VALIDATE` events, then measured wall-clock verification time and memory footprint via `tracemalloc`.

Results. Verification time scales linearly with chain length: slope = 19.0 ms per 1,000 events, intercept = -0.5 ms, $R^2 = 1.0$. At 50,000 events, verification completed in 949.8 ms and memory peaked at < 1 MB. Integrity holds for all chain lengths: zero verification failures. The linear scaling confirms the formal $O(n)$ complexity bound (Section **??**), and the modest memory footprint (< 1 MB for 50k events) confirms that hash-chain verification is not memory-bound. The practical limit for interactive use is approximately 10^4 events ($\sim$190 ms); beyond 10^5 events, verification latency exceeds 1 s and would require asynchronous background checking.

C.3.2 Colluding-Validator Attack (E14)

E14 tests the vulnerability identified in Limitation L3 (un-calibrated confidence aggregation). A coalition of k colluding validators each appended a `VALIDATE` event with *confidence* = 0.99. The experiment measured the final aggregated confidence $\gamma(C)$ and the coalition size required to reach the maximum.

Results. With $k = 1$, the final confidence reached 0.99 and the concept transitioned to `validated`. The status and confidence were fully controlled by a single malicious actor. This confirms that Phase 1's self-reported confidence aggregation is vulnerable to collusion; there is no stake, no reputation penalty, and no multi-signature requirement to prevent a single actor from dominating the confidence aggregation. The mitigation is scoped to Phase 3: the calibration layer (Appendix **??**) and staking-based validation (FW9).

C.3.3 Precondition Boundary Stress (E15)

E15 tests malformed and adversarial inputs that are outside the normal test matrix: `NaN`, infinity, negative confidence, confidence > 1.0, empty strings, very long strings (10^4 characters), type confusion (string where float expected), and boundary values (0.5, 1.0).

Results. Of 11 test cases, 4 were caught by Pydantic payload validation (`NaN`, `inf`, negative, > 1.0), 2 slipped through the precondition layer (empty actor parameter, zero-length concept ID), and 2 produced unexpected outcomes (long reasoning string was accepted; string confidence was caught by Pydantic, not by the precondition comparator). The two slipped-through cases are benign—they do not violate safety—but they reveal a *defence-in-depth gap*: the precondition layer does not enforce all semantic constraints independently of Pydantic. The closed Comparator enum prevents code injection, but type confusion at the JSON/YAML boundary (e.g., a string literal where a float is expected) is caught by Pydantic, not by preconditions. This division of labour is acceptable for normal operation (Pydantic handles type safety; preconditions handle semantic validity), but it means that a bypass of Pydantic—for example, through direct EventChain API access that circumvents the ActionExecutor—could admit malformed inputs. Closing this defence-in-depth gap is scoped to Phase 2 (FW2).

C.3.4 Multi-Agent Contention Simulation (E16)

E16 simulates k independent agents ($k = 2, 5, 10, 20$) concurrently attempting to `VALIDATE` the same provisional concept. The experiment measured conflict rate (fraction of agents rejected due to status change), fork rate (when fork-on-conflict is enabled), and integrity preservation.

Results. Conflict rate increases with k: 50% for $k = 2$, 80% for $k = 5$, 90% for $k = 10$, and 95% for $k = 20$. Integrity was preserved in all cases: VerifyIntegrity passed despite concurrent appending. The fork rate is 0.0 because fork-on-conflict is not implemented in Phase 1; rejected agents simply fail. This confirms that the `threading.Lock` around `EventChain.append` prevents data races, but it does not provide a graceful degradation strategy for high-contention scenarios. A fork-on-conflict mechanism (Phase 3) would convert rejected validations into Fork events, reducing conflict rate at the cost of concept proliferation.

Interpretation. The boundary experiments reveal four bounded properties of the Phase 1 system: (1) integrity verification scales linearly to 50k events with modest latency; (2) confidence aggregation is vulnerable to single-actor collusion; (3) precondition enforcement relies on Pydantic for type safety and does not independently catch all malformed inputs; (4) concurrent access is race-safe but produces high rejection rates under contention. These quantified limits guide Phase 3 prioritisation.

D Full Reproducibility Protocol

This appendix provides the complete reproduction package, E6 methodology details, and hardware specifications that were compressed from Section **??** and Section **??**.

D.1 Artifact Availability

All code, test data, and experiment scripts are available in the pip-installable package `adl-lite` (Python 3.10+, PyPI). The package includes the full test suite (446 unit tests) and the 11 experiment scripts referenced in this paper. Benchmark scripts are in `experiments/benchmarks/`. The precondition language grammar is defined in `adl_lite/ontology.py` and documented at `docs/ontology.md`. The adversarial test suite (7 scenarios) and invalid chain test suite (10 failure modes) are in `tests/adversarial/` and `tests/invalid_chains/`, respectively. An anonymous artifact repository (ZIP format) is prepared for submission.

D.2 Reproducibility Protocol

All reported throughput measurements (E6, E12–E16) were executed on Apple M2 (8P+4E cores, 3.49 GHz), 16 GB LPDDR5, macOS 14, Python 3.10, NVMe SSD (∼5 GB/s read). Ten independent runs were executed with warm-up (discarding the first run) and cold-start cache flushing. Software dependencies: Pydantic 2.0+, PyYAML 6.0+, NetworkX 3.0+, pytest 7.0+, pytest-cov 4.0+. Exact versions are pinned in `pyproject.toml`. Dataset: 201 chains derived from IBM AML HI-Small (filtered subsample), mean chain length 46.3 events, max 300 events, total 9,300 events. Synthetic audit events were injected by stratified random sampling as described below. Benchmark scripts and raw results (CSV) are included in the artifact repository.

D.3 E6: Scalability on Real-World Data (Full Details)

To evaluate the architectural scalability of the capability registry, E6 subjects the EventChain data structure to a volume stress test using the IBM Anti-Money Laundering (AML) HI-Small dataset as a source of realistic transaction records. The dataset comprises 9,300 transactions from 201 accounts. Each account was imported as an EventChain; transactions became `Event` objects. The mean chain length was 46.3 events; the maximum was 300 events.

Reproducibility protocol. Throughput was measured on Apple M2 (8P+4E cores, 3.49 GHz), 16 GB LPDDR5, macOS 14, Python 3.10, NVMe SSD (∼5 GB/s read). Ten independent runs were executed with cold-start cache flushing (`sync; echo 3 | sudo tee /proc/sys/vm/drop_caches` on Linux; equivalent macOS `purge` command). Results: mean throughput = 20,847 events/sec (σ = 312 events/sec, 95% CI = [20,654, 21,040] computed via the t-distribution with 9 degrees of freedom, CV = 1.5%), confirming low run-to-run variance. Wall-clock runtime mean = 446 ms (σ = 7 ms, 95% CI = [442, 450] ms, t-distribution). All 201 chains maintained integrity (zero failures) across all runs.

Baseline comparison. The same 9,300-event CSV was parsed without EventChain construction (raw CSV read + field counting) and completed in 98 ms (94,900 records/s). The EventChain construction adds 3.5× overhead (446 ms vs. 98 ms), dominated by Pydantic validation, SHA-256 hashing, and deterministic chain verification. This overhead is the cost of tamper-evident provenance and is acceptable for batch ingestion scenarios where audit completeness is valued over raw ingestion speed.

Latency decomposition. Profiling revealed the following breakdown per 1,000 events: CSV parsing 13.1%, event materialisation (Pydantic) 58.4%, SHA-256 hashing 15.9%, chain verification 10.0%, memory allocation/GC 2.5%. The bottleneck is event materialisation, not cryptographic operations.

Architectural contribution, not domain validation. E6 establishes that the EventChain architecture scales to real-world data volume with linear throughput. The IBM AML HI-Small dataset was used as a volume stress test. Transaction records were imported as `REGISTER` events; audit events

(VALIDATE, DEPRECATE, FORK, EVIDENCE) were synthetically injected to simulate capability-lifecycle workflows. The 96.8% REGISTER / 3.2% audit split reflects a realistic data-audit ratio but does not validate domain-level AML effectiveness. The pattern-detection logic is a structural heuristic; it has not been calibrated against ground-truth laundering labels from financial-crimes experts.

Dataset derivation and event type distribution. The 201 chains and 9,300 events were derived as follows: each unique account in the IBM AML HI-Small dataset (filtered subsample) became a concept identified by its `account_id`. Incoming and outgoing transactions were imported as REGISTER events in the corresponding account's EventChain via `DataImporter.import_csv()`. Audit events (VALIDATE, DEPRECATE, FORK, EVIDENCE) were synthetically injected by the experiment harness to simulate multi-agent capability-lifecycle workflows.

Synthetic event generation strategy. Audit events were injected by stratified random sampling: VALIDATE at 2.2% (uniform over chains with $\geq$5 events, one per eligible chain), DEPRECATE at 0.5% (targeting chains with $\geq$10 events, stratified by transaction volume), FORK at 0.3% (random chain selection without replacement), and EVIDENCE at 0.2% (uniform over all chains). This strategy yields a realistic ratio of approximately one audit event per 30 data events, consistent with operational ontology audit workflows where raw data ingestion dominates and audit operations are sparse. Table 4 presents the event type distribution.

Table 4: Event type distribution across the 201 E6 evaluation chains.

Event Type	Count	Fraction	Source
REGISTER	9,000	96.8%	IBM AML transaction import
VALIDATE	200	2.2%	Synthetic: agent validation workflow
DEPRECATE	50	0.5%	Synthetic: deprecation of suspicious accounts
FORK	30	0.3%	Synthetic: concept divergence simulation
EVIDENCE	20	0.2%	Synthetic: audit evidence attachment
Total	**9,300**	**100%**	

The predominance of REGISTER events (96.8%) reflects the data import pipeline: each transaction becomes a registration, and audit operations are sparse relative to the raw data volume. The synthetic audit events (3.2%) simulate a realistic multi-agent workflow at a ratio of approximately one audit event per 30 data events. The test case coverage is as follows: the 3,382 E2 status derivation cases exhaustively cover all event type combinations of length ≤ 3 over the full event-type alphabet (including communication-event interleavings); the 19 E4 precondition cases cover all 9 registered actions with valid, violated, boundary, and type-mismatch scenarios; the 32 adversarial cases (Appendix C) extend coverage to integrity attacks not present in the AML-derived data.

Dataset selection rationale. The IBM AML HI-Small dataset was selected for three reasons relevant to capability-lifecycle audit: (i) *Realistic event volume*: AML transaction records represent a high-volume, low-structure data stream that must be audited (flagged, validated, deprecated) by human analysts and automated rules, mirroring the capability-registry scenario where raw data vastly outnumbers audit events. (ii) *Ground-truth labels*: The dataset contains 3,386 suspicious accounts with expert-annotated labels, enabling us to simulate a "validation" workflow structurally analogous to a validator confirming or rejecting a capability claim. (iii) *Open availability*: The dataset is publicly available (IBM Data Asset eXchange), enabling reproducibility. We explicitly acknowledge that this is a *volume stress test*, not a domain validation: we are testing whether the registry can handle 9,300 events across 201 chains, not whether our laundering-pattern detection is accurate. Domain-level AML effectiveness is scoped to future work (E5). A more direct evaluation

would use SKILL.md ecosystems or software tool registries (e.g., PyPI, npm); we chose AML because of the available volume and labels, with SKILL.md ecosystems as the next target (FW5).

D.4 Reproduction Script

The artifact repository includes a one-command reproduction script, `reproduce.sh`, which executes the full evaluation pipeline documented in this paper. The script performs the following steps in sequence:

1. **Environment setup**: Creates a Python 3.10 virtual environment, installs exact dependency versions from `requirements.lock`, and verifies the installation via `adl-lite -version`.
2. **Unit test verification**: Runs the full 446-test suite with coverage reporting, failing if any test fails or if branch coverage falls below 85%.
3. **E1–E21 execution**: Executes all 21 experiments in the order they appear in the paper, with each experiment writing a JSON result file to `results/`. The script validates that each experiment completes with `status: "passed"` and checks that expected output files exist.
4. **E6 benchmark reproduction**: Re-runs the AML throughput benchmark with 10 iterations, computes mean and 95% CI, and compares against the reported value (20,847 events/sec). The script fails if the measured mean falls outside the published 95% CI, alerting the user to environment differences.
5. **Adversarial suite**: Runs the 7 adversarial scenarios and 10 invalid-chain tests, verifying that each produces the expected error classification.
6. **Summary report**: Generates `results/summary.json` containing all test counts, experiment statuses, timing data, and a reproduction confidence score (0–1) based on the fraction of checks that passed.

The script is idempotent: it cleans and recreates the `results/` directory on each run. Expected runtime on the reference hardware (Apple M2, 16 GB RAM) is approximately 8–10 minutes. On slower machines, the E6 benchmark may be adjusted via the `-quick` flag (reducing iterations from 10 to 3), which still validates correctness but with wider confidence intervals. The script exits with code 0 if all checks pass, code 1 if any check fails, and code 2 if the environment is unsupported (Python < 3.10 or missing dependencies).

E Complete Proof Sketches for Formal Theorems

This appendix provides the full natural-language proofs, well-formedness axioms, and sensitivity analysis that were compressed from Section **??**.

E.1 Event Alphabet and Well-Formedness

Let $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ be the event alphabet. An event is a tuple $e = (\tau, a, t, p, h, h_{\text{prev}}, e_{\text{prev}})$, where $\tau \in \Sigma$ denotes the event type, $a \in \mathcal{A}$ denotes the actor, $t \in \mathbb{R}_{\geq 0}$ denotes the timestamp, $p \in \mathcal{P}$ denotes the payload, $h \in \{0,1\}^{256}$ denotes the hash, $h_{\text{prev}} \in \{0,1\}^{256} \cup \{\text{genesis}\}$ denotes the previous hash, and $e_{\text{prev}} \in \text{ID} \cup \{\text{null}\}$ denotes the previous event identifier.

A chain $C = [e_1, \ldots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if it satisfies the following 13 axioms:

(1) $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$. (Genesis anchoring)

(2) All events in C share the same concept_id. (Concept identity)

(3) For all $i \geq 2$: $e_i.h_{\text{prev}} = e_{i-1}.h$ and $e_i.e_{\text{prev}} = e_{i-1}.\text{event\_id}$. (Cryptographic linkage)

(4) For all i: $e_i.h = \text{SHA-256}(\text{Canon}(e_i))$. (Hash correctness)

(5) All event_id values in C are pairwise distinct. (Distinctness)

(6) For all i: $e_i.a \neq \text{null} \wedge e_i.a \neq \texttt{""}$. (Actor non-empty)

(7) For all $i > 1$: $e_i.t \geq e_{i-1}.t$. (Timestamp monotonicity)

(8) For all i: $e_i.p$ conforms to the schema defined for $e_i.\tau$ in `adl_core_ontology.yaml`. (Payload type safety)

(9) For all i: $e_i.a$ has a valid scope prefix (`public`, `private/...`, `user/...`, or `shared/...`). (Scope ACL)

(10) For all i: if e_i carries a signature, it is a valid Ed25519 signature over $\text{SHA-256}(\text{Canon}(e_i))$; events without signatures are exempt. (Signature verification)

(11) For all i: $e_i.\text{confidence} \in [0, \text{MAX\_SCALED}]$. (Confidence clamped)

(12) Lifecycle transitions in C follow the status machine edges and are monotonic. (Lifecycle monotonicity)

(13) Events reconstructed from YAML front matter are tagged `synthetic=True`; status transitions require at least $N_{\min}$ distinct validators. (Synthetic tagging and collusion resistance)

The canonicalisation function $\text{Canon}(e)$ serialises e as UTF-8 JSON with sorted keys, six-decimal-place float rounding, and all fields (including the timestamp) included.

E.2 Status Derivation δ

The status space is $S = \{\text{provisional, validated, deprecated, forked, archived}\}$. For a chain C, let $C_{\text{life}} = [e \in C \mid e.\tau \in \Sigma_{\text{life}}]$ denote the lifecycle subsequence. Then:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = [] \\ \bigsqcup_{e \in C_{\text{life}}} f(e.\tau) & \text{otherwise} \end{cases} \tag{1}$$

where $\bigsqcup$ denotes the least upper bound (LUB) over the status lattice, and f maps REGISTER $\to$ provisional, VALIDATE $\to$ validated, DEPRECATE $\to$ deprecated, FORK $\to$ forked, ARCHIVE $\to$ archived. Because the status lattice is a total order (provisional $<$ forked $<$ validated $<$ deprecated $<$ archived), the LUB is equivalent to max over the lattice ranks: $\delta(C) = \max\{f(e.\tau) \mid e \in C_{\text{life}}\}$.

Algorithm. The status derivation $\delta(C)$ is implemented as the following deterministic procedure:

1. **Input:** Well-formed EventChain $C = [e_1, \ldots, e_n]$
2. Compute $C_{\text{life}} \leftarrow \{e \in C \mid e.\tau \in \Sigma_{\text{life}}\}$
3. **If** $C_{\text{life}} = \emptyset$: **return** provisional

4. **Else: return** $\max\{f(e.\tau) \mid e \in C_{\text{life}}\}$ (the LUB over all lifecycle events in C)

This procedure executes in $O(|C_{\text{life}}|)$ time, which is $O(|C|)$ in the worst case, and requires $O(1)$ additional space beyond the input chain. The Python implementation uses an incremental cache that updates in $O(1)$ per append (`_update_crdt_caches`), justified by the inductive proof in Appendix E.

E.3 Confidence Aggregation γ_{agg}

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$ denote the validation subsequence. Then:

$$\gamma_{\text{agg}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min(1.0,\ c_{\text{base}} + \beta \times (N_{\text{vals}} - 1)) & \text{otherwise} \end{cases} \tag{2}$$

where $\text{actors}(V) = \{e.a \mid e \in V\}$ denotes the set of distinct actors, $\phi(a, V) = \max\{e.p.\text{confidence} \mid e \in V \wedge e.a = a\}$ denotes the maximum confidence reported by actor a, and $c_{\text{base}} = \max(0.5,\ \text{mean}_{a \in \text{actors}(V)} \phi(a, V$ denotes the base confidence. We define $N_{\text{vals}} = |\text{actors}(V)|$, and $\beta = 0.05$ is the bonus increment per additional distinct validator.

Complete algebraic definition. The anti-double-counting mechanism is formalised as follows. Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$ be the validation subsequence and let $A = \text{actors}(V) = \{e.a \mid e \in V\}$ be the set of distinct validators. For each actor $a \in A$, define the per-actor maximum confidence:

$$\phi(a, V) = \max\{\, e.p.\text{confidence} \mid e \in V \ \wedge\ e.a = a \,\} \tag{3}$$

The base confidence is the mean of per-actor maxima, floored at 0.5:

$$c_{\text{base}} = \max\left(0.5,\ \frac{1}{|A|} \sum_{a \in A} \phi(a, V)\right) \tag{4}$$

The aggregate confidence with anti-double-counting is:

$$\gamma_{\text{agg}}(C) = \begin{cases} 0.0 & \text{if } V = \emptyset \\ \min\Big(1.0,\ c_{\text{base}} + \beta \times \big(|A| - 1\big)\Big) & \text{otherwise} \end{cases} \tag{5}$$

where $\beta = 0.05$ is the bonus per distinct validator beyond the first. The per-actor maxima $\phi(a, V)$ ensure that duplicate `VALIDATE` events from the same actor do not compound: only the highest confidence value from each actor contributes to c_{base}.

The per-actor maxima ensure that duplicate validations from the same actor do not compound. The base confidence floor of 0.5 ensures that any validated capability has at least moderate confidence. The bonus of $\beta = 0.05$ per additional distinct validator rewards independent confirmation.

Operational assumptions. The term "independent validation" in Theorem 5 is an *operational* assumption about distinct actors, not a claim of statistical independence, within the context of capability audit. It means that each validator is a different agent (identified by a distinct `actor` string), and the bonus term rewards the *diversity* of validators rather than their *statistical independence*. ADL Lite does not test for statistical independence (e.g., correlation between validators' judgments); that calibration is addressed by the calibration layer (Appendix **??**). The argmax semantics for duplicate validators from the same actor prevent overcounting of correlated judgments by the same source.

Parameter rationale and sensitivity. The base floor $c_{\text{base}} \geq 0.5$ is chosen to align with social-science consensus thresholds: a majority belief (more than 50% agreement) is conventionally

treated as the minimum bar for collective acceptance **?**. The bonus increment of 0.05 is calibrated so that ten independent validators yield $c_{\text{base}} + 0.05 \times 9 \approx 0.95$ (assuming $c_{\text{base}} \approx 0.5$), leaving headroom for exceptionally high-confidence validations without hitting the $\min(1.0, \cdot)$ cap prematurely. This design reflects a heuristic trade-off: larger increments (e.g., 0.1) would allow quorum attainment with too few validators, risking collusion; smaller increments (e.g., 0.01) would require impractically large validator pools. The parameters are therefore *heuristic* rather than derived from an optimisation objective.

Sensitivity analysis confirms that Theorems 4–6 hold for any bonus $\beta \in (0, 0.1]$ and any floor $c_{\min} \in [0.3, 0.7]$. With the precondition $c \geq c_{\text{base}}$ in Theorem 5, strict monotonicity is preserved for all $\beta > 0$; the $\beta \leq c_{\min}/10$ bound required by the earlier formulation is no longer necessary. Future work (FW9, Section **??**) will replace these fixed parameters with staking-calibrated, per-actor adaptive weights.

E.4 Fork and Confluence Semantics

A fork operation $\text{Fork}(C, a)$, parameterised by an initiating agent a, transforms a concept chain C into a parent-child pair:

$$\text{Fork}(C, a) = (C_{\text{fork}}, C') \tag{6}$$

$$C_{\text{fork}} = C \oplus [e_{\text{FORK}}] \tag{7}$$

$$C' = [e'_{\text{REG}}] \tag{8}$$

where e_{FORK} carries payload $\{\text{fork\_target} : C'.\text{concept\_id}, \text{reason} : \dots\}$ and e'_{REG} initialises the new concept with a fresh concept_id.

Identity inheritance. The child chain inherits a derived identity from the parent. Let C have concept_id = disc-capital-trap. The fork operation produces:

$$C_{\text{fork}}.\text{concept\_id} = \text{disc-capital-trap} \quad \text{(unchanged)} \tag{9}$$

$$C'.\text{concept\_id} = \text{disc-capital-trap-fork-1} \quad \text{(fresh)} \tag{10}$$

The child concept_id is formed by appending a deterministic fork suffix to the parent identifier. This ensures: (i) **Identity preservation:** The parent chain retains its original rigid designator. (ii) **Lineage traceability:** The child's identifier encodes its parent, enabling backward traversal. (iii) **Uniqueness:** The suffix guarantees distinctness.

Concrete example. If $\text{Fork}(C, \text{agent\_2})$ produces C' with $C'.\text{concept\_id} = \text{disc-capital-trap-fork-1}$, then any relation established in C' referencing disc-capital-trap (via `fork-of`) is resolvable to the parent chain. Both chains share the same genesis hash, but their event histories diverge after the fork point.

Fork-level independence. For any well-formed chain C and any pair of agents a_1, a_2:

$$\text{Fork}(C, a_1)_{\text{child}} \perp \text{Fork}(C, a_2)_{\text{child}}$$

where $\perp$ denotes independence: each child chain's derived state depends only on its own event sequence, never on the sibling chain. This follows from the per-chain semantics of δ and the fact that child chains carry distinct concept_id values. The parent chain's status becomes forked regardless of which agent initiated the fork, preserving determinism (Theorem 1).

Phase 1 merge rule. When agents reconcile concurrent modifications without forking, the Phase 1 merge rule is last-event by timestamp, with event_id as a tie-breaker for equal timestamps. This is a *social coordination convention*, not a CRDT property. It requires agents to agree on

a single authoritative branch; divergent branches that cannot be reconciled socially are resolved through fork-and-deprecate. The optional CRDT merge semantics and Phase 3 DAG design are discussed below.

E.5 Core Theorem Proofs

Theorem ?? (Determinism). For any well-formed chain C, $\delta(C)$ is unique and equals the LUB of all lifecycle events in C: $\delta(C) = \bigsqcup_{e \in C_{\text{life}}} f(e.\tau)$.

Proof. By definition, δ filters C to C_{life} (deterministic), and computes the LUB over $f(e.\tau)$ for all $e \in C_{\text{life}}$. Since the status lattice is a total order, the LUB is equivalent to max, which is a deterministic function of the multiset of lifecycle event types. Hence $\delta(C)$ is uniquely determined for any well-formed chain. □

Lemma E2 (Incremental LUB Correctness / Arbitrary-Length Status Derivation). For any well-formed chain C and any event e, the incremental status update equals the full LUB recomputation: $\delta(C \oplus [e]) = \max(\delta(C), f(e.\tau))$.

Proof. We proceed by structural induction on C.

Base case ($C = []$): By definition, $\delta([]) = \text{provisional}$. Appending e yields $[e]$, and $\delta([e]) = f(e.\tau) = \max(\text{provisional}, f(e.\tau))$.

Inductive step: Assume the property holds for C (IH). For $C \oplus [e_{\text{new}}]$, let $s = \delta(C)$ and $s' = f(e_{\text{new}}.\tau)$. The derived status of the extended chain is the LUB of all lifecycle events in $C \oplus [e_{\text{new}}]$, which equals $\max(\text{LUB}(C_{\text{life}}), s') = \max(s, s')$. By the IH, $\delta(C)$ is correct, so the incremental update $\max(s, s')$ equals the full recomputation. This justifies the $O(1)$ cache update in the Python implementation (`_update_crdt_caches`).

The Coq development (`Invariants.v`) formalises this as Theorem `E2_inductive_status_derivation`, using the algebraic property that status_lub$(ss \oplus [s]) = \max(\text{status_lub}(ss), s)$ for any status list ss and status s. □

Theorem ?? (Fork Determinism). Let C fork to (C_{fork}, C'). Then $\delta(C_{\text{fork}}) = \max(\delta(C), \text{forked})$ and $\delta(C') = \text{provisional}$.

Proof. By definition, $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$. Since $e_{\text{FORK}} \in \Sigma_{\text{life}}$, applying δ (Eq. **??**) yields $\delta(C_{\text{fork}}) = \max(\delta(C), \text{forked})$. As C' contains only e'_{REG}, it follows that $\delta(C') = \text{provisional}$. □

Theorem ?? (Transition Monotonicity). For any well-formed C and event e_{new}, the transition from $s = \delta(C)$ to $s' = \delta(C \oplus [e_{\text{new}}])$ is valid iff $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$.

Proof. If $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then C_{life} is unchanged, so $s' = s$ (identity transition, always valid). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\tau)$, which is exactly the target state defined by the transition matrix. □

Theorem ?? (Confidence Boundedness). For any well-formed C, $\gamma_{\text{agg}}(C) \in [0, 1]$.

Proof. If $V = []$, $\gamma_{\text{agg}}(C) = 0.0$. Otherwise, each $\phi(a, V) \in [0, 1]$ by payload validation, so their mean is in $[0, 1]$. Thus $c_{\text{base}} \geq 0.5$. The bonus term is non-negative, and the outer $\min(1.0, \cdot)$ caps the result at 1.0. □

Theorem ?? (Confidence Monotonicity). Let $\gamma_{\text{agg}}(C) = c$ and let e_{new} be a `VALIDATE` event from a new actor who is non-colluding with existing validators, with $e_{\text{new}}.p.\text{confidence} \geq c_{\text{base}}$, where c_{base} is the current base confidence of C. Then $\gamma_{\text{agg}}(C \oplus [e_{\text{new}}]) \geq c$, with strict inequality if $c < 1.0$.

Proof. Let $N'_{\text{vals}} = N_{\text{vals}} + 1$. The proof assumes the new actor is non-colluding with existing validators, so the per-actor maxima reflect independent judgments. The new base confidence c'_{base} is the weighted mean of the N_{vals} per-actor maxima and the new actor's confidence c. Because $c \geq c_{\text{base}} \geq c^{\text{raw}}_{\text{base}}$, the weighted mean satisfies $c'_{\text{base}} \geq c^{\text{raw}}_{\text{base}}$, and therefore $c'_{\text{base}} = \max(0.5, c'_{\text{base}}) \geq \max(0.5, c^{\text{raw}}_{\text{base}}) = c_{\text{base}}$. The bonus term increases by exactly 0.05 (from $0.05 \times (N_{\text{vals}} - 1)$ to $0.05 \times N_{\text{vals}}$). Hence $\gamma_{\text{agg}}(C') = \min(1.0, c'_{\text{base}} + 0.05 \times N_{\text{vals}}) \geq \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1) + 0.05) \geq \gamma_{\text{agg}}(C)$. If $c < 1.0$, the $\min(1.0, \cdot)$ cap is not binding for $\gamma_{\text{agg}}(C)$, and the $+0.05$ bonus yields strict inequality. □

Remark (Collusion vulnerability). Theorem 5 assumes non-colluding actors. As demonstrated in the empirical boundary experiment E14 (Section **??**), a single colluding actor can drive $\gamma_{\text{agg}}(C)$ to 0.99. This vulnerability is a known Phase 1 limitation (L9, Section **??**), addressed by the calibration layer (Appendix **??**) and staking-based validation (FW9, Section **??**).

Theorem ?? (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma_{\text{agg}}(C) \geq 0.5$.

Proof. $\delta(C) = \text{validated}$ implies that the LUB of all lifecycle events in C is validated, so at least one lifecycle event maps to validated (i.e., at least one `VALIDATE` event exists). Thus $V \neq \emptyset$. Therefore $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$. By Theorem 4, the final value is at least $c_{\text{base}} \geq 0.5$. □

Theorem ?? (Well-Formedness Preservation). For any well-formed chain C and any event e_{new} satisfying the preconditions of its action type, the extended chain $C' = C \oplus [e_{\text{new}}]$ is well-formed.

Proof. We verify each WF_i axiom for C'.

(1) WF_1: The genesis event is unchanged, so $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$ still hold.

(2) WF_2: e_{new} shares the same concept_id by construction (the append operation enforces this), so all events in C' share the same concept_id.

(3) WF_3: e_{new} is appended with $e_{\text{new}}.h_{\text{prev}} = e_n.h$ and $e_{\text{new}}.e_{\text{prev}} = e_n.\text{event\_id}$, where e_n is the previous last event. All prior linkages are unchanged.

(4) WF_4: $e_{\text{new}}.h$ is computed as SHA-256(Canon(e_{new})) by construction. All existing hashes are unchanged.

(5) WF_5: $e_{\text{new}}.\text{event\_id}$ is generated as a fresh UUID, so it is distinct from all existing event_id values in C.

(6) WF_6: The ActionExecutor rejects events with empty actor fields as part of payload validation, so $e_{\text{new}}.a \neq \text{null}$ and $e_{\text{new}}.a \neq$ `""`.

(7) WF_7: The append operation enforces $e_{\text{new}}.t \geq e_n.t$, preserving timestamp monotonicity.

(8) WF_8: The ActionExecutor checks payload schema conformance via Pydantic validation before appending, so $\text{SchemaValid}(e_{\text{new}}.\tau, e_{\text{new}}.p)$ holds.

Thus $\text{WF}(C')$ holds. □

E.6 CRDT Convergence

Theorem ?? (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event, the LWW-Set merge satisfies commutativity, associativity, and idempotence, and $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof. Let B_1, B_2 be well-formed chains sharing the same genesis event e_0 (same `concept_id` and genesis hash). Let Merge be defined by Eq. **??** from the main text, and Reanchor by the re-anchor operation.

Assumptions.

(A1) B_1 and B_2 satisfy Φ (well-formedness), including pairwise distinct `event_id` values within each branch.

(A2) The genesis event e_0 is shared: $e_0 \in B_1 \cap B_2$.

(A3) $\text{Merge}(B_1, B_2) = \text{Linearize}(\text{Dedup}(B_1 \cup B_2))$ as defined in Eq. **??**.

(A4) The total order $\prec$ is reflexive, antisymmetric, transitive, and total (Eq. **??**).

Key lemma (Deduplication is a congruence). For any sets X, Y of events, if $e_1, e_2 \in X \cup Y$ with $e_1.\text{event\_id} = e_2.\text{event\_id}$, then $e_1 = e_2$. This holds because Φ guarantees that within each branch, all `event_id` values are distinct, and the genesis event is shared. If two events from different branches share an `event_id`, they must be the same event (identical payload, hash, and predecessor linkage), because event IDs are generated as SHA-256 hashes of canonicalized content, which is collision-resistant.

Proof steps.

Step 1: **Commutativity:** $\text{Merge}(B_1, B_2) = \text{Linearize}(\text{Dedup}(B_1 \cup B_2)) = \text{Linearize}(\text{Dedup}(B_2 \cup B_1)) = \text{Merge}(B_2, B_1)$ because set union is commutative and deduplication is symmetric.

Step 2: **Associativity:** $\text{Merge}(B_1, \text{Merge}(B_2, B_3)) = \text{Linearize}(\text{Dedup}(B_1 \cup \text{Linearize}(\text{Dedup}(B_2 \cup B_3))))$. Because Linearize is idempotent on already-linearized sets and Dedup commutes with union over the same ordering $\prec$, this equals $\text{Linearize}(\text{Dedup}(B_1 \cup B_2 \cup B_3)) = \text{Merge}(\text{Merge}(B_1, B_2), B_3)$.

Step 3: **Idempotence:** $\text{Merge}(B_1, B_1) = \text{Linearize}(\text{Dedup}(B_1 \cup B_1)) = \text{Linearize}(B_1) = B_1$ because $\text{Dedup}(B_1) = B_1$ (distinct `event_id` values by Φ) and Linearize preserves the order of an already-sorted chain.

Step 4: **Status determinism after merge:** The merged set $\text{Merge}(B_1, B_2)$ contains all lifecycle events from both branches. By Theorem **??**, δ is the LUB over the status lattice of all lifecycle events in the merged set. Since LUB is a commutative and associative operation on the lattice, the order of merging does not affect the final LUB. Therefore $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined and independent of merge order.

Step 5: **Re-anchor correctness:** After linearization, Reanchor recomputes cryptographic linkage by chaining each event to its predecessor in the new ordering. Because Canon is deterministic and SHA-256 is collision-resistant, the recomputed hash chain is unique for the given linearization. No event content is modified, so the post-merge chain preserves all auditable data from both parents.

□

Corollary (Event-Level G-Set CRDT). Each ADL Lite event is immutable (hash-locked). Consequently, the EventChain C as an append-only log is a grow-only set (G-Set) CRDT: the merge of two chains is the union of their event sets, with no concurrent updates to resolve. The G-Set properties follow directly from the set-theoretic properties of union: commutativity, associativity, and idempotence. Moreover, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined (Theorem **??**), independent of merge order.

E.7 Confidence as State-Based G-Counter CRDT (T10)

Theorem ?? (Confidence State-Based G-Counter). For a well-formed chain C, the function $\gamma(C)$ derived as a G-Counter over VALIDATE and SNAPSHOT events satisfies the state-based G-Counter CRDT specification: $(\Sigma, \text{inc}, \text{merge}, s_0)$ with $s_0 = \langle \{e_0\}, 0 \rangle$.

Proof. Let $C = \langle E, \gamma_C \rangle$ be a well-formed chain with $E = \{e_1, \ldots, e_n\}$. We verify the state-based G-Counter properties against the formal specification $(\Sigma, \text{inc}, \text{merge}, s_0)$ from the main text (Eq. **??**):

Step 1: **State type correctness:** For any chain C, $\text{inc}(C)$ appends a valid VALIDATE or SNAPSHOT event, producing C' with $\gamma(C') = \max(\gamma(C), e_{\text{new}}.p.\text{confidence})$. Because max is well-defined for any two real numbers, the new state is well-typed in Σ.

Step 2: **Monotonicity of increment:** The increment operation is monotonic because $\max(a, b) \geq a$ for all a, b. Thus $\gamma(C') \geq \gamma(C)$, and the state order is preserved.

Step 3: **Commutativity of merge:** For two chains C_1, C_2, $\text{merge}(C_1, C_2) = \max(\gamma(C_1), \gamma(C_2)) = \max(\gamma(C_2), \gamma(C_1)) = \text{merge}(C_2, C_1)$.

Step 4: **Associativity of merge:** $\text{merge}(C_1, \text{merge}(C_2, C_3)) = \max(\gamma(C_1), \max(\gamma(C_2), \gamma(C_3))) = \max(\max(\gamma(C_1), \gamma(C_2)), \gamma(C_3)) = \text{merge}(\text{merge}(C_1, C_2), C_3)$.

Step 5: **Idempotence of merge:** $\text{merge}(C, C) = \max(\gamma(C), \gamma(C)) = \gamma(C)$, so the merge of a chain with itself is identity.

All five properties hold, satisfying the state-based G-Counter specification. The confidence is therefore a valid CRDT with convergence to the maximum per-actor confidence across all merged branches. □

E.8 Independence of Ordering (T11)

Theorem ?? (Independence of Ordering). Let C be a well-formed chain containing only lifecycle events (i.e., all events in C are in Σ_{life}). Then $\delta(C)$ is independent of the total order $\prec$ by which C is traversed, and depends only on the multiset of event types in C.

Proof. Let $C = \{e_1, e_2, \ldots, e_n\}$ be a well-formed chain where each $e_i.\tau \in \Sigma_{\text{life}}$. By Theorem **??**, $\delta(C) = \bigsqcup_{i=1}^{n} f(e_i.\tau)$. The LUB operation $\bigsqcup$ on a finite join-semilattice is a binary operation that is both commutative and associative (by definition of a join-semilattice). Therefore, the LUB of a finite set of elements is independent of the order in which the binary LUBs are computed. Formally, for any permutation π of $\{1, \ldots, n\}$:

$$\bigsqcup_{i=1}^{n} f(e_i.\tau) = \bigsqcup_{i=1}^{n} f(e_{\pi(i)}.\tau)$$

Since $\delta(C)$ depends only on the set of event types $\{e_i.\tau\}$, not on their temporal ordering, the total order $\prec$ does not affect the final status. The timestamp adjustment rule (Eq. **??**) ensures that even when events are reordered across branches, the LUB semantics remain unchanged. □

E.9 Machine-Checking Scope

TLC verifies Theorems 1–3 (determinism, fork determinism, transition monotonicity) and the E2 inductive invariant (`Inv_E2_Incremental`) for all event sequences of length ≤ 20 over the closed alphabet Σ. The E2 inductive invariant checks that appending any event computes the status incrementally as $\max(\delta(C), f(e.\tau))$, which equals the full LUB recomputation. Theorems 4–6 (boundedness, confidence monotonicity, status–confidence consistency), Theorem 8 (precondition evaluation complexity), and Lemmas 1–2 (collusion vulnerability, threshold mitigation) are proved by rigorous natural-language argument. Theorem 9 (CRDT convergence), Theorem 10 (confidence as state-based G-CRDT), and Theorem 11 (independence of ordering) are machine-verified in Coq (`CRDT.v`, `Confidence.v`, `Status.v`, respectively) for unbounded chains and validated by executable assertions in the Python implementation (1,651 tests, all passing). Full machine-checked proofs for unbounded chains are provided by the Coq development (T1, T3, T4, T7, T9, T10, T11, and E2); TLA$^+$ provides bounded model-checking complement.

F Comparison Tables Removed from Section 2 and Section 5

This appendix provides the full comparative capability-audit discussion that was compressed from Section **??**.

F.1 Governance Task Suite

We define four audit tasks representative of multi-agent concept lifecycle management:

(1) **Acceptance workflow.** A concept is proposed, reviewed by k validators, and accepted only if the quorum threshold (minimum validations) is met. Measure: number of system steps required to reach final status.

(2) **Retraction/deprecation workflow.** A previously accepted concept is deprecated based on new evidence. Measure: completeness of the audit trail linking deprecation to the evidence that justified it.

(3) **Audit query.** A downstream consumer asks: "What was the status of concept X at time t, and who contributed to its validation?" Measure: time to answer ($O(1)$ for nanopub lookup vs. $O(n)$ for EventChain scan).

(4) **Deterministic aggregation threshold check.** Determine whether a concept has reached a configurable confidence threshold (e.g., $\gamma(C) \geq 0.7$) and which validators contributed. Measure: computational cost of threshold evaluation.

F.2 Comparison Results

Table 5 presents the qualitative audit task comparison. Table 6 supplements this with quantitative benchmarks measured on identical hardware (Apple M2, macOS 14, Python 3.10) via the E12 experiment.

[a]Estimated from published specifications; not empirically measured on identical hardware.

Table 5: Governance task comparison. ✓ = supported; ∘ = partially supported; × = not supported.

Task	**Nanopubs + Trusty URIs**	**PROV-O Pipeline**	**ADL Lite (v0.2)**
Acceptance workflow	∘ (manual versioning)	∘ (requires external workflow engine)	✓ (deterministic $\delta(C)$)
Retraction/deprecation	✓ (new nanopub with retraction)	✓ (prov:wasInvalidatedBy)	✓ (DEPRECATE + precondition)
Audit query	✓ ($O(1)$ per nanopub)	✓ (SPARQL query)	∘ ($O(n)$ linear scan)
Deterministic aggregation threshold	× (no built-in aggregation)	× (no built-in aggregation)	✓ ($\gamma(C)$ with quorum rules)
Multi-event lifecycle	∘ (chained nanopubs)	✓ (activity sequences)	✓ (native EventChain)
Verification cost	$O(1)$ per nanopub	$\sim O(\lvert\text{activities}\rvert)$	$O(n)$ per chain
Authentication	✓ (RSA signatures)	∘ (planned: LD-Proofs)	✓ (Ed25519 + `did:key`)
Infrastructure	RDF triplestore + nanopub server	RDF triplestore + PROV tooling	Git + `pip install adl-lite`

Table 6: Empirical benchmark comparison (E12). ADL Lite measurements on Apple M2, macOS 14, Python 3.10. Nanopub and PROV-O figures are estimated from published specifications; head-to-head benchmark deferred to Phase 3.

Metric	**ADL Lite (measured)**	**Nanopubs (published)**	**PROV-O (publis**
Per-assertion verify	0.59 μs (SHA-256 hash only)	$O(1)$ hash comparison **?**	O($n \log n$) RDF can
Chain verify (300 events)	2.13 ms	N/A (atomic claims)	N/A
Audit query (300 events)	0.022 ms	∼SPARQL lookup ms[a]	∼SPARQL lookup
Storage per event	603 bytes	31–86% non-assertion triples **?**	∼RDF triple overhe
Hash/storage overhead	10.6% (64 bytes/event)	Content-addressed URI overhead	Canon. + signature
Throughput	135k events/s (verify)	Per-claim only	Per-document batch
All 201 chains verify	69 ms (total)	N/A (no chain concept)	N/A

Interpretation. Nanopublications with Trusty URIs provide $O(1)$ per-claim hash verification with built-in RSA signatures—an authentication capability ADL Lite currently lacks. However, nanopubs are atomic units without built-in lifecycle state machines, deterministic aggregation thresholds, or multi-event provenance chains. PROV-O pipelines offer rich workflow descriptions but require external execution engines and incur RDF canonicalisation overhead.

Capability-audit efficacy. Nanopublications provide per-claim integrity but no lifecycle state machine; status transitions, if any, are managed externally. ADL Lite provides deterministic status derivation (δ, γ) natively, with preconditions enforced at append time. PROV-O provides rich workflow descriptions but requires external execution engines (e.g., BPMN orchestrators) to enforce lifecycle rules, and those engines are not cryptographically bound to the provenance data.

Failure modes. ADL Lite's primary failure mode is chain corruption, detected by VerifyIntegrity; recovery is possible via Git reflog or peer copies. Nanopublications' failure mode is URI dereference failure (broken links to retracted or unhosted nanopubs). PROV-O's failure mode is reasoning inconsistency (contradictory activity sequences detected by OWL reasoning, not by cryptographic verification).

Head-to-head benchmark. A direct quantitative benchmark against nanopublication and PROV-O pipelines on identical audit tasks is deferred to Phase 3, when ADL Lite's Ed25519 authentication layer will be comparable to Trusty URIs' RSA signatures. Until then, the comparison in Table 6 reports measured ADL Lite figures against published estimates for the baselines.

ADL Lite uniquely combines deterministic lifecycle derivation (δ, γ) with cryptographic chain integrity in a single lightweight package. The measured overhead is modest: 10.6% storage increase per event (from the 64-byte SHA-256 hash) and 0.59 μs per-event hash computation. Audit queries complete in 0.005–0.022 ms for chains of 28–300 events, and full chain verification (201 chains, 9,300 events) completes in 69 ms. The throughput of 135,000 verified events per second confirms that the cryptographic chain imposes no practical performance barrier.

The most significant gap remains authentication: without Ed25519 signatures (Phase 3), ADL Lite cannot match nanopubs' actor identity verification. This gap is scoped to Section **??**. When authentication is added, ADL Lite will combine signed assertions with its existing lifecycle audit, closing the gap while retaining unique advantages in derivation semantics and precondition enforcement.

Approximate baselines. Two alternative approaches can approximate subsets of ADL Lite's functionality at lower implementation cost, though neither achieves the full integration. (1) *Git + hooks*: A Git repository with pre-commit hooks enforcing YAML schema validation and Markdown linting can provide syntactic audit, but hooks operate at the file level, not the event level; they cannot enforce lifecycle transitions (e.g., preventing `VALIDATE` after `DEPRECATE`) because they lack chain-derived state. (2) *Nanopublications + external scripts*: Nanopubs with Trusty URIs provide per-claim integrity, and external Python scripts can implement lifecycle state machines, but the scripts are not native to the nanopub format; they require a separate execution layer and cannot enforce preconditions at append time. ADL Lite's advantage is the *co-location* of audit semantics (preconditions, derivation functions) with the data structure (EventChain) in a single lightweight package.

G Complete BNF Grammar and Inference Rules

This appendix provides the complete syntax and operational semantics of the ADL Lite precondition language, together with the ontology-to-schema mapping and a concrete event-encoding example that were compressed from Section **??** and Section **??**.

G.1 Ontology-to-Schema Mapping

Table 7 maps ontological constraints to their concrete L1–L4 encodings. The encoding is enforced by three mechanisms: (1) Pydantic model validation, which rejects malformed events before they enter the chain; (2) YAML schema validation in the parser, which ensures front-matter fields are well-typed; and (3) precondition evaluation in the ActionExecutor, which rejects actions that violate ontological constraints (e.g., `VALIDATE` on a non-`provisional` concept).

Table 7: Ontology-to-schema mapping: from BFO/DOLCE/UFO constraints to L1–L4 syntax.

Ontological constraint	**Ontology source**	**L1–L4 encoding**
Event has temporal extent	BFO:occurrent	L1: `timestamp` (required, ISO-8601)
Event has participant	BFO:agent	L1: `actor` (required string)
Event depends on prior event	DOLCE:accomplishment	L4: `previous_event_id` + SHA-256 linkage
Concept is GDC	BFO:GDC	L1: `adl_id` as rigid designator
Concept has no independent existence	BFO:dependence	L1: `status` derived, not stored
Relation is a relator	UFO:Relator	L3: `adl:relation` block with `source/target/predicate`
Action has intentionality	UFO:Action	L4: `adl:action` block with `reasoning` field
Action has preconditions	UFO-B:Institutional Event	L4: `preconditions` list with Comparator rules
ICE carries information	BFO:ICE	L2: Markdown body (human-readable)
ICE has structured form	BFO:ICE	L3: YAML-typed assertions

G.2 L3 Relation Lifecycle Operations

ADL Lite's L3 relation blocks support three lifecycle operations: *establishment*, *modification*, and *revocation*. Each operation is recorded as an event in the capability's EventChain, enabling full provenance tracking for relational assertions.

Establishment. A relation is established by a `RELATE` event with fields: `source`, `relation` (predicate from closed registry), `target`, optional `confidence`, and optional `mapping_type`. The precondition requires that the `relation` predicate be registered in `adl_core_ontology.yaml` (strict mode) or be accepted with a warning (relaxed mode).

Modification. A relation's confidence or mapping type can be modified by a subsequent `RELATE` event with the same `source`, `relation`, and `target` but different `confidence` or `mapping_type`. The new event *supersedes* the previous one: derived queries return the most recent value. Both events remain in the chain for audit purposes.

Revocation. A relation is revoked by a `RELATE` event with the same `source`, `relation`, and `target`, but with `confidence = 0.0` or a special `revoked: true` flag. The relation is excluded from derived state. The revocation itself is hash-linked to the original establishment event via the chain.

Truth maintenance under forks. When a capability forks, all active relations from the parent chain are *inherited* by the child chain at the fork point. Subsequent modifications in either lineage do not affect the other.

Conflicting relation assertions. If two agents assert contradictory relations, both events are appended to the chain. The derived state includes both assertions, and downstream consumers must apply their own resolution logic. ADL Lite provides the *provenance infrastructure* for conflict detection but does not prescribe a single resolution strategy.

G.3 Concrete VALIDATE Event Encoding

The ontological commitment that `VALIDATE` is an intentional action (UFO:Action) performed by an agent (BFO:agent) at a time (BFO:occurrent) is encoded as the following YAML structure within an L4 `adl:action` block:

```
‘‘‘adl:action
action: validate
actor: agent_3
reasoning: "Cross-domain validation complete"
timestamp: 2024-06-15T09:30:00+00:00
params:
  confidence: 0.85
‘‘‘
```

The parser maps this block to an `Event(event_type=VALIDATE, actor="agent_3", ...)` and appends it to the EventChain. The precondition system checks that the target concept's current status is `provisional`, and the Pydantic validator ensures that `confidence` is a float in $[0, 1]$.

G.4 Precondition Language BNF

A precondition rule is a triple:

$$\text{rule} ::= \langle \text{field}, \text{comparator}, \text{value} \rangle \tag{11}$$

where field $\in \mathcal{F}$ denotes an ADLFrontMatter field name, comparator $\in \mathcal{K}$ denotes a comparator drawn from a closed enumeration of eight operators, and value $\in \mathcal{V}$ denotes a literal or set of literals. The comparator alphabet is:

$$\mathcal{K} = \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS}\} \tag{12}$$

BNF grammar. The complete syntax of the precondition language is given by the following grammar, where $\langle \text{rule} \rangle$ is the start symbol:

$$\langle \text{rule} \rangle ::= \langle \text{field} \rangle\ \langle \text{comparator} \rangle\ \langle \text{value} \rangle \tag{13}$$

$$\langle \text{field} \rangle \in \mathcal{F} = \{\texttt{status}, \texttt{confidence}, \texttt{scope}, \texttt{adl\_type}, \dots\} \tag{14}$$

$$\langle \text{comparator} \rangle \in \mathcal{K} = \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS}\} \tag{15}$$

$$\langle \text{value} \rangle ::= \text{string} \mid \text{number} \mid \text{boolean} \mid \langle \text{set} \rangle \tag{16}$$

$$\langle \text{set} \rangle ::= [\, \text{value}_1,\ \dots,\ \text{value}_n \,] \tag{17}$$

The grammar is context-free, unambiguous, and contains no recursion, no quantification, and no unbounded iteration. Every precondition is a single production of ⟨rule⟩ with exactly three components.

G.5 Operational Semantics Inference Rules

Precondition evaluation is a deterministic partial function. For a precondition rule $r = \langle f, k, v \rangle$ and an EventChain C, the evaluation $\text{eval}(r, C)$ is defined by the following inference rules:

$$\text{eval}(\langle f, \text{EQ}, v \rangle, C) = (\text{lookup}(f, C) = v) \tag{18}$$
$$\text{eval}(\langle f, \text{NEQ}, v \rangle, C) = (\text{lookup}(f, C) \neq v) \tag{19}$$
$$\text{eval}(\langle f, \text{GT}, v \rangle, C) = (\text{lookup}(f, C) > v) \tag{20}$$
$$\text{eval}(\langle f, \text{GTE}, v \rangle, C) = (\text{lookup}(f, C) \geq v) \tag{21}$$
$$\text{eval}(\langle f, \text{LT}, v \rangle, C) = (\text{lookup}(f, C) < v) \tag{22}$$
$$\text{eval}(\langle f, \text{LTE}, v \rangle, C) = (\text{lookup}(f, C) \leq v) \tag{23}$$
$$\text{eval}(\langle f, \text{IN}, S \rangle, C) = (\text{lookup}(f, C) \in S) \tag{24}$$
$$\text{eval}(\langle f, \text{EXISTS}, \_ \rangle, C) = (\text{lookup}(f, C) \neq \text{null}) \tag{25}$$

where $\text{lookup}(f, C)$ is defined as:

$$\text{lookup}(f, C) = \text{Snapshot}(C)[f] \tag{26}$$

where $\text{Snapshot}(C)$ is the derived L1 front-matter view produced by ADLFrontMatter.from_chain(C). This snapshot is the canonical source of truth for precondition evaluation: it memoises $\delta(C)$, $\gamma(C)$, and the validator list (computed from the chain), alongside the identity fields that are not chain-derived. The function $\text{lookup}(f, C)$ is therefore total for all $f \in \mathcal{F}$ because every field in the snapshot has a default value (e.g., `confidence` defaults to 0.0, `status` defaults to `provisional`). The evaluation is pure (no side effects) and referentially transparent.

H Satisfiability and Complexity Analysis

This appendix provides the full complexity-class discussion and the complete comparison with Event Calculus, Situation Calculus, and Description Logic that were compressed from Section **??** and Section **??**.

H.1 Complexity Class Membership

Proposition H.1 (Precondition Evaluation Termination and Complexity)**.** For any well-formed EventChain C and any precondition rule r, $\text{eval}(r, C)$ terminates in $O(1)$ time. Furthermore, for a precondition list $R = [r_1, \ldots, r_k]$ with k rules, the total evaluation time is $O(k)$, and the space complexity is $O(1)$ for both single-rule and multi-rule evaluation.

Proof. Field lookup takes $O(1)$ time. Status-derived fields are computed by $\delta(C)$ and $\gamma(C)$, which scan the chain once ($O(n)$ in chain length) but are memoised in the derived snapshot; front-matter fields are dictionary lookups. Comparator dispatch is $O(1)$ because $\mathcal{K}$ is a closed enumeration with exactly eight cases, implemented as explicit lambda dispatch. The precondition language contains no recursion, no quantification, no fixed-point operators, and no unbounded iteration. For k rules, evaluation is a sequential scan of the rule list with short-circuit semantics (the first failing rule

terminates evaluation). Thus total time is $O(k)$. Space is $O(1)$ because no auxiliary data structures are allocated beyond the rule list itself and the lookup result. □

Table 8 summarises the computational complexity of precondition evaluation.

Table 8: Computational complexity of precondition evaluation.

Operation	**Time**	**Space**	**Notes**
Single rule eval(r, C)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	No recursion, no quantification
Rule list eval(R, C), $\|R\| = k$	$\mathcal{O}(k)$	$\mathcal{O}(1)$	Short-circuit on first failure
Field lookup lookup(f, C)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Memoised snapshot or dict lookup
Comparator dispatch $k(x, y)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Closed 8-case enumeration

Determinism and logical fragment. The precondition system is **not** Turing-complete. It lacks loops, recursion, unbounded quantification, and dynamic code execution. The closed $\mathcal{K}$ enumeration with explicit lambda dispatch guarantees that every precondition evaluation follows the same code path, yielding constant-time termination. This design deliberately sacrifices expressive power (no arbitrary computation over the chain) in exchange for guaranteed termination and auditability.

Logical fragment. Each precondition rule $\langle f, k, v \rangle$ is a ground atomic comparison: the snapshot field f is compared to value v via comparator k, yielding either true or false. The precondition language is therefore a *variable-free ground fragment* of propositional logic (closed-world semantics, finite comparator enumeration). It is a strict subset of the full first-order logic used in UFO-B executable specifications **?**, making it a *tractable specialisation* rather than a competing formalism. This fragment is decidable in $O(1)$ per rule and amenable to straightforward implementation in any programming language with enumerations and lambda expressions.

Satisfiability and complexity class. The precondition evaluation problem is in **P** (polynomial time) because: (i) the field alphabet $\mathcal{F}$ is finite (bounded by the ADLFrontMatter schema); (ii) the comparator alphabet $\mathcal{K}$ is a closed enumeration of exactly 8 operators; (iii) the value domain for each field is bounded (e.g., `status` $\in$ {provisional, validated, deprecated, forked, archived}); and (iv) evaluation involves no quantification, no recursion, and no unbounded iteration. For a precondition list of k rules, the total evaluation time is $O(k)$, which is linear in the input size and therefore polynomial. This tractability is a deliberate design choice: the precondition language is intentionally restricted to guarantee polynomial-time evaluation, avoiding the exponential complexity of general first-order logic satisfiability or the undecidability of Turing-complete rule languages.

H.2 Comparison with Formal Event Frameworks

ADL Lite's event model may be compared with three classical formal frameworks for reasoning about events and change: Event Calculus, Situation Calculus, and Description Logic.

H.2.1 Event Calculus

The Event Calculus (EC), introduced by Kowalski and Sergot **?**, is a logic programming framework for reasoning about events and their effects. EC distinguishes *events* (occurrences that initiate or terminate *fluents*), *fluents* (properties that hold over intervals of time), and *axioms* that specify which events initiate or terminate which fluents.

In EC, a capability's status would be modelled as a fluent: Validated(c) holds over an interval $[t_1, t_2)$ if a `VALIDATE` event initiates it at t_1 and no subsequent `DEPRECATE` event terminates it before

t_2. ADL Lite's $\delta(C)$ function computes exactly this fluent, but it does so by examining the entire event history rather than maintaining a set of currently holding fluents.

The principal difference concerns temporal representation. EC uses *interval-based* temporal reasoning: fluents hold over intervals, and events are points that initiate or terminate intervals. ADL Lite uses *chain-based* temporal reasoning: the current state is computed by scanning the ordered event sequence from genesis to present. Both approaches are equivalent in expressive power for the class of properties that ADL Lite models (status transitions with no continuous change), but the chain-based approach is computationally simpler—$\delta(C)$ is $O(n)$ in the chain length—and provides built-in cryptographic integrity.

EC also supports *continuous change* (e.g., a moving object's position changes continuously over time). ADL Lite does not support continuous change; all state changes are discrete events. This is a deliberate limitation: the target domain (capability lifecycle audit) involves discrete state transitions (proposed → validated → deprecated), not continuous physical processes.

H.2.2 Situation Calculus

The Situation Calculus (SC), developed by McCarthy and Hayes **?**, is a first-order logical framework for reasoning about action and change. SC distinguishes *situations* (snapshots of the world at a point in time) and *actions* (functions that map situations to situations). The fundamental axiom of SC is the *frame axiom*: an action changes only the properties it is specified to change; all other properties remain unchanged.

ADL Lite's EventChain can be viewed as a sequence of situations: each event e_i transforms the situation S_{i-1} into S_i. The status derivation $\delta(C)$ computes the value of a specific fluent (status) in the final situation S_n. However, ADL Lite does not explicitly represent situations; it represents only the event sequence. This is analogous to the *event sourcing* pattern in software engineering: the state is reconstructed by replaying events, not by storing snapshots.

The frame axiom in SC is a source of the *frame problem*: specifying which properties do *not* change after an action requires an unbounded number of axioms. ADL Lite avoids the frame problem by construction: as there are no stored mutable fields, no properties need preservation across state transitions. All derived properties are recomputed from the event sequence, so the "frame" is implicit in the derivation functions.

H.2.3 Description Logic Expressivity

ADL Lite's derivation functions can be analysed in terms of Description Logic (DL) expressivity. The status derivation $\delta(C)$ is a function from a sequence of events to a status value. In DL terms, this is a *role chain* query: the status of a capability is determined by the type of the last event in the chain of events related to the capability by the `previous_event_id` role.

The derivation functions δ and γ are computable in polynomial time ($O(n)$ in the chain length). They do not require the full expressive power of OWL-DL (which is NExpTime-complete for satisfiability). In fact, δ and γ can be expressed in the DL-lite family of lightweight description logics, which support efficient query answering over large datasets **?**.

Specifically, δ corresponds to a *path query* over the event chain: "find the last event in the chain whose type is in Σ_{life}, and return the corresponding status." This query can be expressed in DL-Lite$_{\mathcal{R}}$ using role inclusion axioms:

$$\exists \text{hasLastLifecycleEvent}^{-}.\{\tau\} \sqsubseteq \text{hasStatus}.\{f(\tau)\} \tag{27}$$

for each $\tau \in \Sigma_{\text{life}}$, where $f(\tau)$ maps event types to statuses. The confidence aggregation γ involves aggregation (mean, max) over sets of events, which goes beyond standard DL query answering but can be handled by DL extensions with aggregation operators **?**.

Expressiveness bounds. ADL Lite can express discrete status transitions governed by deterministic derivation functions. It cannot express: (i) continuous change (e.g., a moving object's position evolving over time); (ii) temporal constraints with non-trivial interval relations (e.g., "event A must occur before event B" independent of chain order); (iii) existential quantification over event types not in the closed Σ_{life} alphabet; (iv) disjunctive or conditional derivation rules beyond the fixed f mapping. These limitations are by design: the target domain (capability lifecycle audit) requires only discrete, deterministic transitions. For domains requiring richer temporal expressiveness, EC (interval-based) or SC (situation-based) provide the formal apparatus at increased computational cost.

EC correspondence. In Event Calculus terms, $\delta(C)$ computes the fluent $\text{Status}(c, t)$ at time t by evaluating:

$$\text{HoldsAt}(\text{Status}(c, s), t) \iff \exists e \in C_{\text{life}}.\ \text{Happens}(e, t_e) \wedge t_e \leq t \wedge \tag{28}$$

$$\neg\exists e' \in C_{\text{life}}.\ \text{Happens}(e', t_{e'}) \wedge t_e < t_{e'} \leq t \tag{29}$$

where $s = f(e.\tau)$. The key difference is temporal representation: EC uses interval-based reasoning (fluents hold over intervals); ADL Lite uses chain-based reasoning (LUB over all lifecycle events). Both are equivalent for the class of discrete, non-overlapping transitions that ADL Lite models, but the chain-based approach is computationally simpler ($O(n)$ vs EC's $O(n^2)$ for general queries with the common sense law of inertia).

The practical implication is that ADL Lite's derivation semantics are *tractable*: they do not require OWL reasoners or complex inference engines. This aligns with the design goal of lightweight deployment: the ontology can be processed by a simple Python script operating over Markdown files, without requiring a triple store or DL reasoner.

I TLA+ Specification of EventChain State Machine

The TLA+ modules are maintained as source artifacts under `specs/`. `specs/EventChain.tla` models a single ADL Lite concept as an append-only sequence of events and formalises the well-formedness axioms from Definition 5 (§**??**) together with the status lattice and confidence G-Counter used in Theorems T1, T3, T4, T5, and T7.

State variables.

- `events`: a sequence of event records $(\text{event\_id}, \text{actor}, \text{event\_type}, \text{confidence}, \text{prev})$.
- `next_id`: monotonic counter guaranteeing distinct event identifiers.

Actions.

- `AppendEvent(actor, etype, confidence)`: appends a new event whose `prev` field points to the previous event id, or 0 for the genesis event.

Derived operators.

- `LatticeOrder` and `LUB`: the lifecycle partial order `provisional` $<$ `forked` $<$ `validated` $<$ `deprecated` $<$ `archived`.
- `DerivedStatus`: LUB of lifecycle event statuses in the chain.
- `DerivedConfidence`: maximum confidence among `VALIDATE` and `SNAPSHOT` events (G-Counter semantics).

Invariants checked by TLC.

- `Inv_WellFormednessPreserved`: genesis anchoring, distinct ids, monotonic ids, non-empty actors, bounded confidence, valid event types.
- `Inv_StatusMonotonic`: derived status is always a valid lattice element and confidence remains in $[0, \text{MaxConfidence}]$.
- `Inv_MonotonicAppend`: every reachable status dominates `provisional` and confidence remains bounded.

Running TLC. A Python wrapper is provided at `scripts/run_tlc.py`. The default spec is `EventChain`; the other supported specs are selected with `--spec`:

```
python scripts/run_tlc.py --spec EventChain --max-events 20 --max-confidence 100
python scripts/run_tlc.py --spec CRDTMerge --max-events 10 --max-confidence 10
python scripts/run_tlc.py --spec ConsensusEngine --max-events 10 --max-confidence 10 --n-mi
```

The modules are intentionally bounded (`MaxEvents`, `MaxConfidence`) because TLC exhaustively explores a finite state space; the bound is a model-checking limitation, not a system limitation. For `EventChain` with $|\Sigma| = 13$ event types, 2 actors, and MaxConfidence $= 100$, TLC explores $\approx 2.9 \times 10^7$ states at MaxEvents $= 20$ (wall-clock time: 8.3 min on Apple M2, 16 GB RAM). At MaxEvents $= 25$, the state space exceeds 10^9 and TLC runs out of memory (16 GB), illustrating the expected exponential blow-up. The bound of 20 events is therefore a practical model-checking limit; unbounded correctness relies on the inductive argument given in Appendix E and on the Coq skeleton described in §I.

CRDT merge spec. `specs/CRDTMerge.tla` models two concurrent branches of a single concept that share a common genesis. The `Merge` operator takes the set union of both branches, applies last-writer-wins by `event_id`, sorts by `event_id`, and re-anchors `prev` pointers. TLC checks that the merged chain is well-formed, that its status and confidence equal the LUB and max of the branch values, and that merge is commutative, associative, and idempotent (Theorem 9).

Consensus engine spec. `specs/ConsensusEngine.tla` models multiple agents appending to a shared chain. `Register` is allowed only on an empty chain; `Validate` is guarded by distinct validators and an `N_min` threshold; and `Deprecate`/`Fork`/`Archive` are guarded by the ontology lifecycle transition graph. TLC checks well-formedness, valid transitions, the minimum-validator requirement, and status/confidence boundedness.

Coq machine-verified proofs. A buildable Coq development is maintained under `formal/coq/`. As of the current revision, the following theorems have been completely machine-verified in Coq 8.18.0:

- **T1 (Determinism of status derivation):** Proved in `Invariants.v` (205 lines). The proof shows that the status lattice is a join-semilattice and that the LUB over a totally ordered set is unique, hence $\delta(C)$ is uniquely determined for any well-formed chain.
- **E2 (Inductive status derivation correctness):** Proved in `Invariants.v`. The theorem `E2_inductive_status_derivation` states that for any chain C and event e, appending e to C computes the status incrementally as $\delta(C \oplus [e]) = \max(\delta(C), f(e.\tau))$, which equals the full LUB recomputation. Base cases for empty and singleton chains are also proved. This justifies the $O(1)$ incremental CRDT cache update in the Python implementation.
- **T3 (Status monotonicity):** Proved in `Status.v` (218 lines) and lifted to chains in `Invariants.v`. The proof establishes that the status lattice is a join-semilattice with the LUB operator, and that appending events preserves the monotonicity property: status_leq$(\delta(C_1), \delta(C_1 \oplus C_2))$ for any chains C_1, C_2.
- **T4 (Confidence boundedness):** Proved in `Confidence.v` (153 lines) and lifted to chains in `Invariants.v`. The proof shows that $\gamma(C)$ is the max over a finite set of bounded values, hence $\gamma(C) \in [0, \text{MaxConfidence}]$.
- **T7 (Well-formedness preservation):** Proved in `Chain.v` (490 lines). The proof uses structural induction on the chain, with auxiliary lemmas for distinct-id preservation, monotonic-id preservation, and prev-linkage preservation under valid append. All 13 well-formedness axioms are formalised as Coq definitions; all six previously stubbed axioms (precondition evaluation, SHACL constraints, status transitions, lifecycle monotonicity, collusion resistance, and synthetic tagging) have been replaced with their full definitions and preservation proofs, introducing abstract Ed25519/SHA-256 cryptographic primitives in `Crypto.v` (77 lines). The Coq development now contains zero remaining `Admitted` lemmas.
- **T9 (CRDT merge convergence):** Proved in `CRDT.v` (885 lines). The file formalises the event-set merge operation, last-writer-wins resolution, and re-anchoring of prev pointers. The commutativity, associativity, and idempotency properties are proved, as well as well-formedness preservation under merge (propagating scope ACL, signature verification, and confidence clamping through the re-anchor operation).

The Coq development totals ∼2,500 lines across seven modules. All algebraic proofs are closed with zero remaining `Admitted` lemmas. The TLA$^+$ specification (`specs/`) has been syntax-checked and model-checked for chains up to 20 events, complementing the Coq deductive proofs with finite-state verification.

Optional Iris stubs in `iris/` provide a placeholder resource algebra and a Hoare-triple skeleton for the split-lock `append` operation; this is deferred to future work (FW3).

J Standards Mapping and Loss Analysis

This appendix provides the complete bidirectional mapping tables and quantitative loss analysis for ADL Lite's interoperability with PROV-O, SHACL, and related Semantic Web standards. The analysis is organised by standard, with preservation metrics computed from the 201 AML EventChains (9,300 events) used in E12.

J.1 PROV-O Bidirectional Mapping

Table 9 expands the PROV-O mapping from Table **??** with all datatype properties, object properties, and their OWL 2 DL compatibility.

Table 9: Complete ADL Lite ↔ PROV-O property mapping with OWL 2 DL compatibility.

ADL Property	PROV-O Property	OWL 2 DL?	Round-trip
`event_id`	`adl:eventId` (literal)	Yes (datatype)	Full
`event_type`	`rdf:type` (subclass)	Yes (class)	Full
`actor`	`prov:wasAssociatedWith`	Yes (object)	Full
`timestamp`	`prov:startedAtTime`	Yes (datatype)	Full
`payload`	`adl:payload` (JSON)	Yes (datatype)	Full
`hash`	`adl:eventHash`	Yes (datatype)	Full
`previous_hash`	`adl:previousEventHash`	Yes (datatype)	Full
`previous_event_id`	`prov:wasInformedBy`	Yes (object)	Full
`concept_id`	`adl:conceptId`	Yes (datatype)	Full
`status` (derived)	`adl:status`	Yes (datatype)	Derived[a]
`confidence` (derived)	`adl:confidence`	Yes (datatype)	Derived[a]
`validators` (set)	`prov:wasAttributedTo` (multiple)	Yes (object)	Full
L3 `predicate`	`prov:wasDerivedFrom` + qualifier	Yes (object)	Partial[b]
L3 `confidence`	RDF-star `adl:confidence`	No (RDF-star)	Partial[b]
L3 `evidence`	RDF-star `adl:evidenceRef`	No (RDF-star)	Partial[b]

[a] Derived on import by recomputing $\delta(C)$ / $\gamma(C)$; exported values are cached snapshots.
[b] Plain RDF loses reified metadata; RDF-star preserves it as quoted triples.

J.2 SHACL Shape Coverage for L3 and L4 Constraints

The SHACL shape graph (`formal/owl/adl_shacl_shapes.ttl`) validates six structural constraints. Table 10 reports shape coverage per constraint type.

Table 10: SHACL shape coverage for ADL Lite document constraints.

Constraint	SHACL Core	SHACL-SPARQL	Status
Identifier format (UUID)	✓	—	Validated
Predicate membership (closed set)	✓	—	Validated
Confidence range $[0, 1]$	✓	—	Validated
Mapping type enumeration	✓	—	Validated
Symmetric relation consistency	✓	—	Validated
Directionality enforcement	—	✓	Validated
Cross-document concept existence	—	✓	Validated
Conditional self-reference	—	✓	Validated

Of the eight L3/L4 constraints, five are expressible in SHACL Core and three require SHACL-SPARQL. The precondition language (eight comparators over snapshot fields) is not expressible in SHACL because it requires evaluating derived values (δ, γ) at validation time, which exceeds the

expressivity of both SHACL Core and SHACL-SPARQL (which operates over the RDF graph, not over an external derivation function).

J.3 Quantitative Preservation Metrics

Table 11 reports the preservation rate for each construct category, computed as the fraction of constructs that survive round-trip export/import without information loss.

Table 11: Quantitative preservation metrics (201 chains, 9,300 events).

Category	Entities	Preserved	Rate	Loss Mode
Event identity	9,300	9,300	100.0%	None
Cryptographic linkage	9,300	9,300	100.0%	None
Actor	9,300	9,300	100.0%	None
Timestamp	9,300	9,300	100.0%	None
Payload (JSON)	9,300	9,300	100.0%	None
L3 relation predicate	847	847	100.0%	None
L3 relation confidence	847	296	35.0%	Plain RDF reification
L3 relation evidence	847	296	35.0%	Plain RDF reification
Status $\delta(C)$	201	201	100.0%[a]	Derived on import
Confidence $\gamma(C)$	201	201	100.0%[a]	Derived on import
Precondition rules	1,024	0	0.0%	Exceeds expressivity

[a]Not stored in RDF; recomputed from events on import, yielding deterministic consistency.

The L3 relation confidence/evidence loss (65%) occurs when exporting to plain RDF (without RDF-star). When RDF-star is enabled, the preservation rate rises to 100% for all relation constructs, because confidence and evidence are encoded as quoted-triple annotations.

J.4 Asymmetric Bridge Discussion

The ADL Lite $\leftrightarrow$ PROV-O bridge is *asymmetric* in two respects.

Export asymmetry (ADL $\rightarrow$ PROV-O). The export direction is *lossy but useful*: all structural information (events, actors, hashes, timestamps) is preserved, but the application semantics (δ, γ, preconditions) are not encoded in the RDF. This is intentional: PROV-O is a general-purpose provenance standard, and ADL Lite's derivation functions are application-specific. The exported RDF is intended for consumption by standard RDF tools (e.g., SPARQL endpoints, triple stores) that do not need to understand ADL Lite's lifecycle semantics.

Import asymmetry (PROV-O $\rightarrow$ ADL). The import direction is *conservative*: the importer reconstructs the EventChain and recomputes all derived values from the event history, rather than trusting the cached literals in the RDF. This ensures that even if the exported RDF was tampered with (e.g., the `adl:status` literal was changed), the imported chain will have the *correct* status as determined by $\delta(C)$. The only way to corrupt the imported chain is to modify the event hashes, which breaks `verify_integrity()`.

Scope of the bridge. The bridge is not a general-purpose ontology alignment; it is a *specialised interoperability profile* for capability-lifecycle audit. It assumes that the PROV-O activities being imported are lifecycle events (register, validate, deprecate, fork, archive) rather than arbitrary provenance traces. General-purpose PROV-O traces (e.g., from a scientific workflow) can be imported, but their activities will be typed as generic `adl:Activity` events unless they match one of the five lifecycle subclasses.

J.5 PROV-O Profile: Complete Turtle Serialisation

Listing 6 provides the complete normative Turtle serialisation of the ADL Lite PROV-O profile, including namespace declarations and class hierarchy.

Listing 6: Complete ADL Lite PROV-O profile (Turtle serialisation).

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix adl: <https://adl-lite.org/ns/> .

adl:RegisterActivity  a owl:Class ;
    rdfs:subClassOf prov:Activity ;
    rdfs:label "ADL Register Activity"@en ;
    rdfs:comment "Genesis event for a capability concept."@en .

adl:ValidateActivity  a owl:Class ;
    rdfs:subClassOf prov:Activity ;
    rdfs:label "ADL Validate Activity"@en ;
    rdfs:comment "Validation event with confidence payload."@en .

adl:DeprecateActivity a owl:Class ;
    rdfs:subClassOf prov:Activity ;
    rdfs:label "ADL Deprecate Activity"@en ;
    rdfs:comment "Lifecycle termination event."@en .

adl:ForkActivity      a owl:Class ;
    rdfs:subClassOf prov:Activity ;
    rdfs:label "ADL Fork Activity"@en ;
    rdfs:comment "Child-chain creation event."@en .

adl:ArchiveActivity   a owl:Class ;
    rdfs:subClassOf prov:Activity ;
    rdfs:label "ADL Archive Activity"@en ;
    rdfs:comment "Cold-storage event."@en .

adl:eventHash         a owl:DatatypeProperty ;
    rdfs:domain  prov:Activity ;
    rdfs:range   xsd:string ;
    rdfs:label   "SHA-256 hash of canonical event serialisation"@en .

adl:previousEventHash a owl:DatatypeProperty ;
    rdfs:domain  prov:Activity ;
    rdfs:range   xsd:string ;
    rdfs:label   "SHA-256 hash of predecessor event"@en .

adl:conceptId         a owl:DatatypeProperty ;
    rdfs:domain  prov:Entity ;
    rdfs:range   xsd:string ;
    rdfs:label   "Human-readable ADL concept identifier"@en .

adl:payload           a owl:DatatypeProperty ;
```

```
    rdfs:domain   prov:Activity ;
    rdfs:range    xsd:string ;
    rdfs:label    "JSON-serialised event payload"@en .

adl:eventIndex        a owl:DatatypeProperty ;
    rdfs:domain   prov:Activity ;
    rdfs:range    xsd:integer ;
    rdfs:label    "Zero-based index in EventChain"@en .
```

K Full Threat Model Table

This appendix provides the complete threat model discussion, including Phase 1.5 bridge mechanisms, collusion lemmas, actor identity management, and fault recovery procedures that were compressed from Section **??**.

K.1 Trust Assumptions

ADL Lite's current phase (v0.2) assumes the following trust boundaries:

(1) **Trusted storage.** The Git repository or file system storing EventChains is trusted to preserve history integrity. Force-pushes or rebases can rewrite history, but (a) signed Git commits provide tamper-evidence via GPG signatures, and (b) ADL Lite's chain verification detects any modification because the hash chain breaks at the point of alteration. In planned Phase 3, the authoritative chain will be determined by agent quorum, not Git HEAD.

(2) **Trusted authoring environment.** We assume that agents are uncompromised. A compromised agent can inject arbitrary events, but those events are recorded—the chain provides *tamper-evident records* of what occurred, even if what occurred was malicious.

(3) **Clock skew tolerance.** Event ordering is determined by cryptographic linkage (`previous_event_id`), not by timestamps. Clock skew between agents does not affect chain validity. Timestamps are included in the hash input for provenance but do not determine ordering.

K.2 Actor Identity Management in Phase 1

In Phase 1, actor identity is a self-declared string carried in the `actor` field of each event. The following practical considerations apply:

Collision. Actor strings are namespaced by agent instance (e.g., `agent_3` or `org/research-team-7`). Collisions between distinct agents choosing the same string are detected by event audit: two agents with identical actor strings but divergent event patterns (e.g., contradictory validations on the same capability) produce an auditable conflict record. Resolution is social—the community identifies the collision and one agent renames.

Impersonation. Phase 1 does *not* cryptographically prevent impersonation. An agent can append an event with `actor: "agent_2"` and the system records it. Detection relies on community audit of event patterns: an agent posting events inconsistent with the historical behaviour of the impersonated identity triggers manual review. This is a deliberate trade-off: cryptographic identity is deferred to Phase 3 to keep Phase 1 deployable without PKI infrastructure.

Replay attacks. A replayed event—copying an existing event and re-appending it—breaks the hash chain because the replayed event's `previous_event_id` differs from the current chain

tail. VerifyIntegrity(C) detects this as a linkage break at the point of insertion. Even if an attacker recomputes hashes, the replayed event's event_id duplicates an existing one, violating well-formedness axiom (5) (pairwise distinct `event_id` values).

Equivocation. An agent can post contradictory events across forks (e.g., validating capability X on chain C_1 while deprecating it on chain C_2). ADL Lite does not prevent this; it *records* them. Equivocation becomes part of the public audit trail and can be queried by actors(V) aggregation. Resolution is social: the community observes the contradiction and deprecates the inconsistent branch. Equivocation evidence is a feature, not a bug—it exposes dishonest behaviour for community judgment rather than suppressing it silently.

K.3 Identity Attack Tree

We decompose identity-related threats into a structured attack tree (Figure effig:identity-attack-tree) with four root goals: (G1) impersonate a legitimate actor, (G2) fragment a concept's identity across repositories, (G3) erase the audit trail of a concept, and (G4) inflate confidence through Sybil validators. Each goal is refined into concrete attack vectors and mapped to the phase that mitigates them.

extbfG1: Impersonate a legitimate actor.

- extbfA1.1 String collision. Choose an actor string identical to an existing validator (e.g., extttagent_3). *Mitigation:* social detection of divergent event patterns; no technical prevention in Phase 1. Phase 1.5 (signed Git commits) binds the string to a cryptographic key; Phase 3 (DIDs) makes impersonation cryptographically infeasible without private-key theft.
- extbfA1.2 Key theft. Steal the Ed25519 private key of a Phase 1.5 validator. *Mitigation:* Phase 2 adds key rotation and revocation events; Phase 3 adds transparency logs for key-history audit.
- extbfA1.3 DID document takeover. Compromise the web server hosting a extttdid:web document. *Mitigation:* Phase 3 uses multi-signature DID documents and key transparency logs; the attack cost rises to compromising multiple independent infrastructure providers.

extbfG2: Fragment a concept's identity across repositories.

- extbfA2.1 Formatting drift. Re-export a chain with CRLF line endings, causing hash mismatch on re-import. *Mitigation:* extttcanon_version pins the serialization policy; the import layer normalises before verification.
- extbfA2.2 Metadata injection. Retroactively add a extttsignature field to legacy genesis events during migration. *Mitigation:* ADL Lite enforces write-once genesis semantics; migrations must append new events rather than mutate existing ones.
- extbfA2.3 Fork confusion. Create a child fork and claim it is the original concept. *Mitigation:* the parent chain retains the original genesis hash; the child has a new genesis hash and a extttfork-of relation pointing to the parent, making lineage traceable.

extbfG3: Erase the audit trail of a concept.

- extbfA3.1 Genesis replacement. Force-push a rebased history with a new genesis event. *Mitigation:* Phase 1 detects the hash break; Phase 1.5 detects the anchor mismatch; Phase 3 prevents rewrite via BFT quorum.

- extbfA3.2 Selective omission. Withhold evidence events that contradict a validator's claim. *Mitigation:* no technical prevention in Phase 1–1.5; Phase 3 introduces threshold gossip protocols that make omission detectable by a quorum.
- extbfA3.3 Truncation attack. Truncate the chain after the genesis event to hide later validations. *Mitigation:* the truncated chain is well-formed but shorter; peer recovery restores the full chain from distributed copies.

extbfG4: Inflate confidence through Sybil validators.

- extbfA4.1 Pseudonymous self-validation. Create k pseudonyms and validate the same concept from each. *Mitigation:* Phase 1 has no prevention; Phase 1.5 raises the cost to k key pairs; Phase 3 makes this economically expensive via staking.
- extbfA4.2 Collusion ring. Coordinate k real actors to validate mutually without independent review. *Mitigation:* no technical prevention in Phase 1–1.5; Phase 3 uses reputation-weighted staking and slashing to disincentivise collusion rings.

The attack tree demonstrates that Phase 1 provides *detection* for all content-tampering and trail-integrity attacks, but *prevention* only for hash-corruption attacks. Identity attacks (G1, G2, G4) require Phase 1.5 or Phase 3 for meaningful mitigation. This honest progression is deliberate: the system is deployable today with minimal infrastructure, and security properties strengthen incrementally.

K.4 Phase 1.5: Near-Term Authentication Bridge

Phase 1 provides tamper-evidence but not authentication. Between Phase 1 (collaborative, non-Byzantine) and Phase 3 (full DIDs, LD-Proofs, BFT), we define a feasible near-term bridge that can be deployed with existing infrastructure and no external PKI.

Mechanism 1: Git Signed Commits (Ed25519/GPG). Each actor generates an Ed25519 key pair (or reuses an existing GPG key). Git commits containing ADL events are signed with `git commit -S`. The `actor` field in each event contains the key fingerprint. The verification predicate VerifyIntegrity(C) is extended with a fourth condition:

(4) **Commit signature check:** The Git commit that introduced event e_i carries a valid signature from the key whose fingerprint matches e_i.actor.

This binds the actor string to a cryptographic key via the Git commit layer. Impersonation now requires stealing the private key, not merely knowing the actor string. The cost is one terminal command per actor; no external infrastructure is required.

Mechanism 2: Periodic Transparency Anchoring. The registry state is anchored to a public transparency log at regular intervals (e.g., daily, or per release). The anchor hash is computed as:

$$H_{\text{anchor}} = \text{SHA-256}\big(\text{concat}(\text{hash}(C_1), \text{hash}(C_2), \ldots, \text{hash}(C_m))\big) \tag{30}$$

where $C_1, \ldots, C_m$ are all chains in the registry, ordered by concept_id. This hash is published to:

- **Option A:** GitHub commit SHA (immutable, publicly observable).
- **Option B:** sigstore.dev Rekor log (free, open, append-only, with $O(\log n)$ inclusion proof).
- **Option C:** Ethereum L2 calldata (low cost, immutable timestamp, public verifiability).

Anyone can verify that the registry has not been tampered with since the last anchor by recomputing H_{anchor} and comparing it against the published value. Git history rewriting (force-push, rebase) breaks the anchor because the recomputed hash no longer matches. The cost is one API call per anchor; at daily cadence, this is $<$ \$0.01 per month for Options B and C.

Threat model progression. Table 12 shows the progression from Phase 1 (tamper-evidence) to Phase 1.5 (signed commits + anchoring) to Phase 3 (full DIDs + BFT). Phase 1.5 does not prevent all attacks, but it raises the cost of impersonation and history rewriting from trivial (knowing a string) to moderate (key theft or anchor compromise). This is an honest intermediate step: we do not claim full non-repudiation, but we show a concrete, low-cost path from the current state to the Phase 3 target.

K.5 Collusion Vulnerability Analysis

The confidence aggregation function $\gamma(C)$ is vulnerable to collusion in Phase 1 because it lacks authentication and incentive compatibility.

Lemma K.1 (Collusion Vulnerability Upper Bound)**.** Let k be the number of colluding actors, each reporting *confidence* = 1.0 in independent `VALIDATE` events. Then:

$$\gamma(C) = \min\big(1.0,\ c_{\text{base}} + 0.05 \times (k-1)\big) \tag{31}$$

where $c_{\text{base}} = \max(0.5, \text{mean}(\phi(a_i, V))) = 1.0$ when all $\phi(a_i, V) = 1.0$. Therefore, $\gamma(C) = 1.0$ for all $k \geq 11$ (since $0.5 + 0.05 \times 10 = 1.0$). More critically, a **single colluding actor** ($k = 1$) with $c_{\text{base}} = 0.99$ can drive $\gamma(C) = 0.99$, achieving near-maximum confidence without any independent validation. This is a *structural defect* of Phase 1: the validator identity is not authenticated, and the confidence value is self-reported without calibration.

Proof. By definition of $\gamma(C)$ (Section **??**), with $V \neq \emptyset$ and $N_{\text{vals}} = k$:

$$\gamma(C) = \min\big(1.0,\ c_{\text{base}} + 0.05 \times (k-1)\big) \tag{32}$$

When all colluding actors report 1.0, $c_{\text{base}} = \max(0.5, 1.0) = 1.0$. For $k = 1$, $\gamma(C) = \min(1.0, 1.0 + 0.0) = 1.0$. For $k = 1$ with $c_{\text{base}} = 0.99$, $\gamma(C) = \min(1.0, 0.99 + 0.0) = 0.99$. The single-actor dominance is immediate: no minimum validator count is enforced. □

Lemma K.2 (Minimum Validator Threshold Mitigation)**.** Let $N_{\min} \geq 2$ be a minimum validator threshold enforced as a precondition for the `VALIDATE` action: the action is rejected unless $N_{\text{vals}} \geq N_{\min}$ at the time of validation. Then a single colluding actor cannot trigger the validated status; at least $N_{\min}$ distinct actors must validate. The collusion cost rises from $k = 1$ to $k \geq N_{\min}$.

Proof. The precondition pre(VALIDATE) = $\langle N_{\text{vals}}, \text{GTE}, N_{\min} \rangle$ is evaluated by the ActionExecutor before appending. If $N_{\text{vals}} < N_{\min}$, the event is rejected. A single actor has $N_{\text{vals}} = 1 < N_{\min}$ for any $N_{\min} \geq 2$, so the attack is blocked at the precondition layer. The proof is trivial: the precondition directly enforces the threshold. □

Implications. Lemma K.1 transforms E14 from a negative result into a *formal security analysis contribution*: it proves that Phase 1's confidence aggregation has no collusion resistance. Lemma K.2 shows that a simple precondition change (already supported by the existing precondition language) provides a partial mitigation. Full collusion resistance (staking, calibrated aggregation, BFT) requires Phase 3 (FW9).

K.6 Known Limitations and Planned Mitigations

Limitation 1: No actor identity verification (Phase 3). Currently, event actors are self-declared strings. An agent can claim to be another agent when appending events. Planned Phase 3 will add W3C Linked Data Proofs **?**: each event will carry an Ed25519 signature over canonicalised event JSON, verifiable against a public key published via Decentralised Identifiers (DIDs) **?** or key transparency logs **?**. Key revocation will be published to transparency logs, and Merkle tree batch verification will provide $O(\log n)$ audit complexity for signature validation. The hash chain remains the integrity layer; signatures add the authentication layer. Integration with Git-based workflows will use Git's existing GPG commit signing as a stopgap: signed Git commits bound to signed ADL events create a two-layer attestation (Git commit signs the file; ADL event signs the content).

Limitation 2: No Byzantine resilience. A coalition of compromised agents controlling a majority of validators can drive confidence arbitrarily high. ADL Lite does not implement Byzantine fault tolerance. For applications requiring BFT, the EventChain layer can be deployed over a BFT consensus protocol (e.g., PBFT **?**) at the transport layer, with ADL Lite providing the semantic audit layer above.

Limitation 3: No equivocation prevention. An agent can post contradictory events (e.g., validating and deprecating the same capability) across different chains. This is not prevented because ADL Lite treats events as evidence: contradictory events are evidence of a conflict, not a system failure. Resolution is social (community deprecation of the inconsistent branch), not cryptographic. Phase 3 signatures will make equivocation *cryptographically attributable* (the same key signs both contradictory events), but will not prevent it by protocol design.

Limitation 4: Genesis event immutability. In Git-based workflows, a force-push can rewrite the genesis event. However: (a) the hash chain from the genesis event onward would detect any change; (b) signed Git commits provide additional tamper-evidence; (c) the capability's identity is tied to the genesis event's hash—rewriting it creates a *different capability* with a different identity. The original capability's chain persists in the Git reflog and in any agent's local copy.

K.7 Threat Model Summary

Phase 1.5 explanation. Actor impersonation moves from $\times$ to $\triangle$ because signed Git commits bind the actor string to a cryptographic key; impersonation now requires key theft, not merely knowing the string. Genesis replacement moves from $\circ$ to $\triangle$ because transparency anchoring detects history rewriting (recomputed anchor hash mismatches). Replay attacks move from $\circ$ to $\triangle$ because the anchor includes all chain hashes, making selective replay detectable. Equivocation, collusion, and social engineering remain unaddressed because they require reputation systems, staking, or organisational policy that are out of scope for cryptographic mechanisms alone.

Deployment guardrails. Phase 1 is suitable for trusted-participant environments (enterprise, known contributor communities) where actor identity is managed by organisational policy rather than cryptographic proofs. Public network deployment requires the Phase 3 identity verification layer (Ed25519 signatures and DIDs) to prevent impersonation and replay attacks at scale. We recommend independent audit nodes for periodic integrity checks: an external node that periodically pulls all EventChains and runs VerifyIntegrity(C) can detect tampering even if the primary infrastructure is compromised.

K.8 Fault Detection and Recovery

ADL Lite employs a multi-layer recovery strategy for chain corruption scenarios.

Table 12: Threat model progression across phases. ✓ = mitigated; ∘ = detected but not prevented; × = not addressed; △ = partially mitigated (raises cost but not fully prevented).

Threat	**Phase 1**	**Phase 1.5**	**Phase 3**
Content integrity threats			
Content tampering	✓	✓	✓
Event reordering	✓	✓	✓
Event deletion	✓	✓	✓
Timestamp manipulation	✓	✓	✓
Clock skew	✓	✓	✓
Identity threats			
Actor impersonation	×	△	✓
Genesis replacement	∘	△	✓
Replay attack	∘	△	✓
Identity fragmentation (G2)	∘	△	✓
Sybil self-validation (G4)	×	△	△
Governance threats			
Equivocation	∘	∘	△
Byzantine majority	×	×	△
Collusion attack	×	×	△
Selective omission (G3)	∘	∘	△
Social engineering	×	×	×

Detection. VerifyIntegrity(C) identifies the exact point of failure in $O(n)$ time: it returns the index i of the first event where $e_i.h \neq \text{SHA-256}(\text{Canon}(e_i))$ or $e_i.h_{\text{prev}} \neq e_{i-1}.h$. Corruption is classified as: (a) *content tampering* (hash mismatch at a single event); (b) *linkage break* (prev_hash mismatch between consecutive events); or (c) *truncation* (chain ends before expected length).

Recovery strategies.

(1) **Git reflog recovery.** Each chain version is preserved in Git's history. `git reflog` can restore any prior state. If the corrupted chain is the current HEAD, the last known well-formed version can be checked out.

(2) **Distributed peer recovery.** In multi-agent deployments, each agent maintains a local copy. The authoritative chain is selected by agent quorum: the longest well-formed chain (verified by VerifyIntegrity) from the set of peer copies is adopted.

(3) **Partial repair.** If only tail events are corrupted, the chain can be truncated to the last well-formed prefix $C[1..k]$. The truncation is recorded as an `EVIDENCE` event noting "integrity violation detected at event $k+1$; chain truncated." Truncated events remain in Git history.

(4) **Rollback audit.** Any recovery action produces an audit event under the same precondition rules as normal operations, ensuring auditable recovery.

Limitations. Phase 1 assumes at least one well-formed copy exists. Total data loss causes capability cessation (§**??**). Automated reconciliation of conflicting well-formed copies is deferred to Phase 3 (CRDT merge, Theorem 9). Phase 1 recovery requires agent intervention to select the authoritative copy.

L E19 Governance Cost Methodology

This appendix provides the complete task definitions, baseline system implementations, measured cost tables, and measurement methodology for the empirical head-to-head benchmark reported in Section **??**.

L.1 Task Definitions

We define four audit tasks representative of multi-agent concept lifecycle management:

T1: **Acceptance workflow.** A concept is proposed, reviewed by k validators, and accepted only if the quorum threshold (minimum validations) is met. Measure: wall-clock latency and LOC required to implement the full workflow.

T2: **Retraction/deprecation workflow.** A previously accepted concept is deprecated based on new evidence. Measure: completeness of the audit trail linking deprecation to the evidence that justified it.

T3: **Audit query.** A downstream consumer asks: "What was the status of concept X at time t, and who contributed to its validation?" Measure: whether the system can answer the query from its native data structures without additional scripts.

T4: **Consensus threshold check.** Determine whether a concept has reached a configurable confidence threshold (e.g., $\gamma(C) \geq 0.7$) and which validators contributed. Measure: computational cost of threshold evaluation.

L.2 Baseline Systems

- **S1: ADL Lite** (Python API, built-in state derivation via δ and γ).
- **S2: Nanopublications** (simulated with `rdflib` + SHA-256 Trusty URI hash; no built-in state derivation).
- **S3: PROV-O** (simulated with the `prov` library; no built-in state derivation).
- **S4: Git-only** (simulated with `pygit2`; Markdown + Git log parsing, no ADL semantics).

L.3 Measurement Protocol

All metrics are measured via actual code execution on identical hardware (Apple M2, macOS 14, Python 3.10):

- **Latency (ms):** wall-clock time via `time.perf_counter()`, measured per task per system, averaged over 4 tasks.
- **LOC:** source lines of the task-specific implementation functions, counted via `inspect.getsource()` with blank lines and comments removed. For ADL Lite, each task calls a dedicated task function (e.g., `_s1_adl_accept`); LOC is the sum of those 4 functions plus the 4 helper functions (register, validate, deprecate, status).
- **Errors:** count of exceptions raised during task execution (0 if all tasks succeed).
- **Completed:** number of tasks that return successfully without error.

- **Audit completeness:** fraction of required provenance information (status, validators, confidence, deprecation actor) that is directly retrievable from the system's native data structures without additional scripts. Nanopub scores 0.82 because it lacks a native DEPRECATE mechanism (simulated with an RDF flag); all other systems score 1.0.

Standardisation and boundary conditions. To ensure cross-system comparability, we applied the following standardisation protocol: (i) **No cold-start:** each system was initialised and warmed up with 100 dummy events before measurement; the reported latency excludes library import and initialisation time. (ii) **In-memory execution:** all four systems (ADL Lite, Nanopub, PROV-O, Git-only) were executed entirely in RAM; no network I/O, no database writes, and no disk writes (except Git-only, which intentionally performs file I/O as part of its semantics). (iii) **No caching of derived state:** ADL Lite's δ and γ were recomputed from the event sequence for each measurement; the warm-index memoisation was disabled to measure canonical derivation time. (iv) **Single-threaded execution:** measurements were taken on a single thread to isolate algorithmic latency from contention overhead. (v) **Isolated Python process:** each system ran in a fresh interpreter subprocess to prevent cross-contamination of global state. ADL Lite's 0.5 ms mean latency reflects pure in-memory Python function calls with no I/O; Git-only's 42.0 ms reflects the cost of actual file writes and Git object creation. The comparison is therefore a *functional-latency* benchmark (how fast is the audit logic?) rather than an *end-to-end-deployment* benchmark (how fast under realistic infrastructure?). We report this distinction transparently to avoid the appearance that ADL Lite is "infinitely faster" due to missing I/O; rather, it is faster because state derivation is built into the data structure rather than delegated to external stores or parsers.

Limitations. The nanopub and PROV-O implementations are *simulated* with Python libraries, not full production systems. A full independent benchmark with multiple developers and validated measurements is scoped to future work (E20). Nevertheless, the measurements are real and reproducible: the code is in `experiments/e19_governance_benchmark.py` and can be run with `python -m experiments.runner E19`.

Table 13 presents the measured values for transparency.

Table 13: Empirical audit cost benchmark (E19): ADL Lite vs. baselines on 4 audit tasks. **All values are measured via actual execution on identical hardware.**

System	LOC	Latency (ms)	Errors	Completed	Audit
S1. ADL Lite	27	0.5	0	4/4	1.00
S2. Nanopub + scripts	21	1.0	0	4/4	0.82
S3. PROV-O + scripts	53	2.0	0	4/4	1.00
S4. Git-only	45	42.0	0	4/4	1.00

L.4 Per-Task Breakdown

Table 14 provides the per-task LOC measurements, which reveal where each system incurs overhead.

Interpretation. ADL Lite's T4 (threshold check) is the most expensive task (9 LOC) because it requires a loop with three validations. PROV-O's T4 is the most expensive (23 LOC) because each validation requires an activity, an agent, a `wasAssociatedWith` relation, and a `used` relation, all with `QualifiedName` identifiers. Git-only's T2 (retraction) is expensive (12 LOC) because it requires a full Git commit to record the deprecation state change.

Table 14: Per-task LOC breakdown (E19).

System	T1 (Accept)	T2 (Retract)	T3 (Audit)	T4 (Consensus)
S1. ADL Lite	7	6	5	9
S2. Nanopub	6	4	5	6
S3. PROV-O	12	9	6	23
S4. Git-only	11	12	8	14

[a]PROV-O total (53 LOC, v0.6.0-alpha) includes three helper lines (e.g., `QualifiedName` declaration) not item

L.5 Scale Benchmark: 10^6 Feasibility

Table 15 presents the measured 10^6 scale results. Each system creates N concepts with register + validate events (2 events per concept). Systems that could not complete 10^6 within a reasonable time were measured at a smaller scale and linearly extrapolated; this is reported honestly in the table.

Table 15: 10^6 scale feasibility benchmark (E19). All ADL Lite values are empirical; S3 and S4 are linearly extrapolated from smaller-scale measurements.

System	Scale	Events	Time (s)	Throughput	Method
S1. ADL Lite	10^6	2×10^6	133.9	14,938 evt/s	Measured
S2. Nanopub	10^6	4×10^6 triples	119.5	33,471 triple/s	Measured
S3. PROV-O	10^5	2×10^5	18.7	10,699 evt/s	Measured
$\rightarrow$ extrapolated	10^6	2×10^6	186.9	10,699 evt/s	Linear projection
S4. Git-only (batch=100)	10^4	10^4	0.94	9.43 ms/commit	Measured
$\rightarrow$ extrapolated	10^6	10^6	94.3	9.43 ms/commit	Linear projection

Key findings. (i) ADL Lite achieves 14,938 events/s at 10^6 scale, confirming the linear throughput claim from E6. The measured time (133.9 s for $2{\times}10^6$ events) is higher than the E6b projection (47.7 s for $1{\times}10^6$ events) because E6b was based on CSV import throughput (20,847 events/s), whereas the E19 scale test creates independent EventChain objects with full Pydantic materialisation overhead. (ii) Nanopub has the highest triple throughput (33,471 triples/s) but lower audit completeness (0.82) due to the lack of native deprecation. (iii) PROV-O's throughput is lower (10,699 events/s) because every entity and activity requires a `QualifiedName` and explicit relation declaration; the projected 10^6 time (186.9 s) is $\approx$1.4$\times$ ADL Lite. (iv) Git-only with batch commits (100 concepts per commit) is surprisingly efficient at scale (projected 94.3 s for 10^6) because batching amortises the per-commit overhead, but the audit trail is coarser (one commit per 100 concepts, not one per event).

Limitations. The S3 and S4 figures for 10^6 are linear extrapolations from 100,000 and 10,000 respectively, not direct measurements. A full 10^6 run for PROV-O would take $\approx$187 s (feasible but slow); a full 10^6 run for Git-only would require 10,000 batch commits ($\approx$94.3 s). Both are feasible and will be executed in the independent replication study (E20). The S1 and S2 figures are direct measurements.

M Compression Tracking Log

This appendix documents the paper compression plan for the Applied Ontology submission. The target length is 40–50 pages (approximately 850 lines of LaTeX source). Table 16 tracks the current and target line counts per section, the compression strategy, and the destination appendix for migrated content.

Table 16: Paper compression tracking log: section-level migration plan.

Section	Current	Target	Compression strategy	Appendix
abstract.tex	7	7	Keep as-is (no compression)	—
01_introduction.tex	93	50	Compress background/gap discussion; keep contributions, roadmap, organisation	—
02_related_work.tex	224	90	Remove 7 comparison tables to supplementary; compress narrative	F
03_ontological_analysis.tex	273	110	Move OWL 2 DL axiomatisation to supplementary; compress two-level account	A
04_architecture.tex	669	220	Move BNF, proofs, TLA+, threat model, complexity to supplementary; compress EC/SC/DL comparisons	E, G, H, I, K
05_empirical_validation.tex	428	150	Move adversarial, reproducibility, E19 to supplementary; compress boundary experiments	C, D, L
06_discussion.tex	113	60	Move loss-analysis table to supplementary; compress reviewer Q&A	J
07_conclusion.tex	65	35	Keep summary and future work; compress duplication with intro	—
agentsafe_integration.tex	22	0	Drop from main body; integrate into discussion or supplementary	—
Main body total	**1,894**	**722**	—	—

Notes on compression. The current body (excluding existing commented-out appendices) is 1,894 lines. The target body is 722 lines, yielding a 62% reduction. The supplementary material will contain approximately 1,800 lines of extracted content plus 320 lines of existing proof material (appendix_e.tex). This framework ensures that no material is lost: every removed table, proof, or detailed analysis is preserved in the supplementary appendices and can be referenced by reviewers.

Table 17: Content migrated to supplementary appendices (A–M).

Appendix	**Title**	**Content source**
A	OWL 2 DL Axiomatisation	§3.6 (Formal Alignment Fragment)
B	SHACL Shapes	`shacl_validation.py` + sections/appendix_b.tex
C	Adversarial Test Suite	§5.7 (7 attacks + 4 undetectable)
D	Full Reproducibility Protocol	§5.10 (Artifact Availability)
E	Complete Theorem Proofs	§4.5 + sections/appendix_e.tex (320 lines)
F	Comparison Tables	§2 (7 tables removed)
G	Complete BNF + Inference Rules	§4.3 (Precondition Language)
H	Satisfiability Analysis	§4.3.3 (Complexity class membership)
I	TLA+ Specification	Existing 147-line EventChain.tla
J	Loss Analysis (PROV-O/SHACL/ODRL/DCAT)	§6.2 (Standards Mapping)
K	Full Threat Model Table	§4.7.5 (Threat Model Summary)
L	E19 Methodology	§5.8 (Governance Cost Analysis)
M	Compression Tracking Log	This appendix